

Situational Restaurant Recommendation via Web Crawling and LLM-Driven Review Analysis

상황 맞춤형 맛집 추천 : 네이버 지도 웹 크롤링과 LLM 기반 리뷰 분석 활용

“Where2Eat”

김도윤 김효민 이서현
이세연 임령아 장한나 한가은



목차

01 프로젝트 전체 과정

02 주제 정의 및 필요성

03 데이터 수집

04 데이터 전처리 및 적재

05 데이터 분석

- LLM 기반 집약 & 상황정의 생성

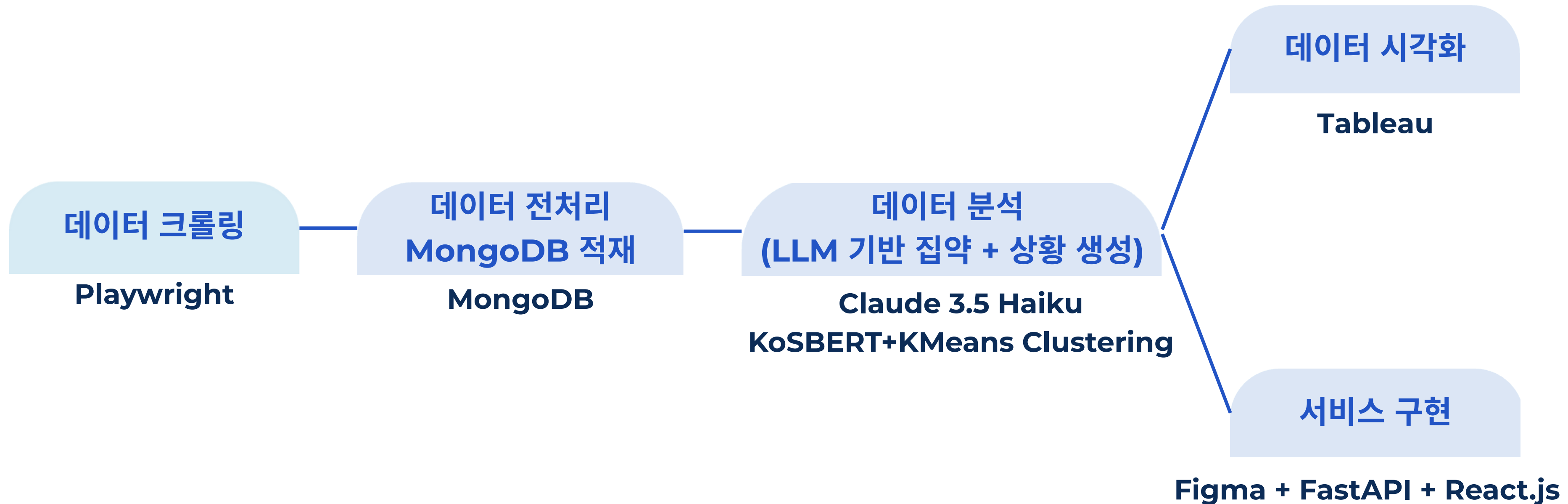
06 데이터 시각화

07 서비스 구현

01

프로젝트 전체 과정

- 프로젝트 과정도



• 역할 분담

데이터 수집 (크롤링)

김도윤 이서현(메인)
김효민 이세연 임령아 장한나 한가은

데이터 전처리 + 적재

임령아 장한나 한가은

데이터 분석 (LLM 기반)

김효민 이세연 한가은

데이터 시각화

임령아 장한나

서비스 구현

사이트 기획 : 장한나 한가은
사이트 제작 : 김도윤 이서현

프로젝트 일정표

3월						
일	월	화	수	목	금	토
16	17	18	19	20	21	22
			회의 1			
23	24	25	26	27	28	29
			회의 2 주제 선정	각자 역할 결정		각자 역할 활용 기술 조사
30	31					
4월						
일	월	화	수	목	금	토
		1	2	3	4	5
		1번과정 마감		2번과정 마감	피드백 1	
6	7	8	9	10	11	12
3번과정 마감 / 1차(테스트) 회의				1번 과정 마감	서비스 구현 방식 회의	
13	14	15	16	17	18	19
2번 과정 마감					피드백 2	
20	21	22	23	24	25	26
3번과정 마감 / 2차 (전체) 회의						
27	28	29	30			
사이트 기획 마감	사이트 제작		회의3 2차 역할 분담			

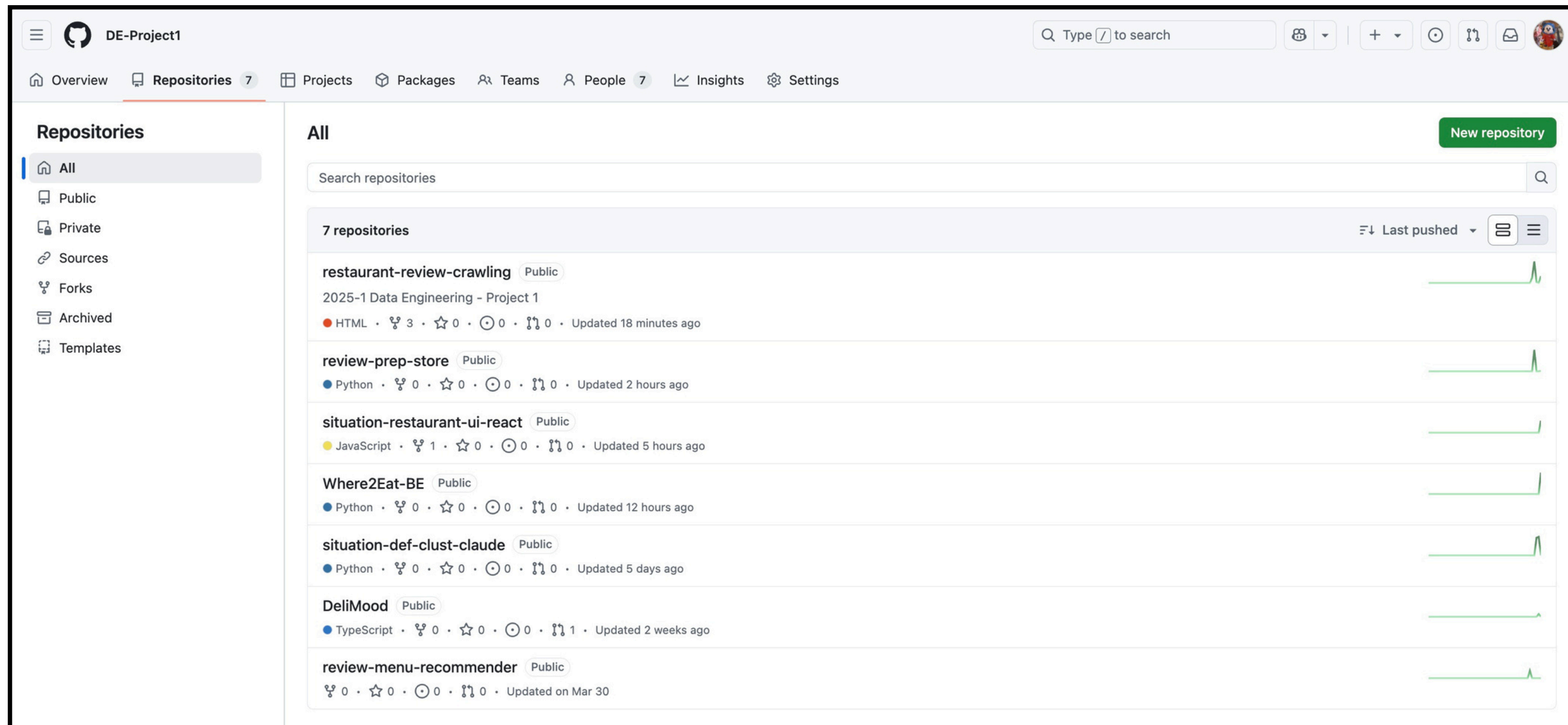
5월						
일	월	화	수	목	금	토
				1	2	3
4	5	6				
시각화, 사이트, 피피티 완성 회의		제출 마감				

• 진행 과정 with Notion

☰ 리스트 📅 표 +		📅 표 📅 타임라인 📅 표 +		📄 회의 유형	Aa 이름		
자료실	📄 Streamlit 참고 자료	회의록 및 일정		📄 전처리&D...	📄 MongoDB Atlas 사용법	분석팀	📄 3차
	📄 깃허브 사용법 & 협업하기			📄 시각화&개...	📄 1차	분석팀	📄 프롬프트 설계
	📄 서현) 레퍼런스 북마크용			📄 크롤링팀	📄 2차	분석팀	📄 단계별 구성
	📄 크롤링 실행하기			📄 전처리&D...	📄 2차	시각화&개...	📄 시각화
	📄 크롤링 진행 분담			📄 피드백	📄 TA 세션 1	시각화&개...	📄 데이터 시각화
	📄 공용 아이디 정보			📄 피드백	📄 TA 1 Session 피드백	분석팀	📄 대표 상황정의 생성
	📄 크롤링 실행하기			📄 전체 회의	📄 0404 전체 회의	전체 회의	📄 전체 회의
				📄 전처리&D...	📄 1차 팔로업 문서		
				📄 분석팀	📄 1차 팔로업 문서	시각화&개...	📄 데이터 시각화
				📄 크롤링팀	📄 1차 팔로업 문서	크롤링팀	📄 Raw data crawling + 분담
				📄 전체 회의	📄 0406 전체 회의	시각화&개...	📄 백엔드 API 명세서
				📄 피드백	📄 0406 팀별 안건 정리	제출마감	
				📄 크롤링팀	📄 S3에 저장		
				📄 전체 회의	📄 0412 전체 회의		
				📄 분석팀	📄 2차		
				📄 전처리&D...	📄 2차 팔로업 문서		
				📄 피드백	📄 TA 2 (팀원용)		
						2025년 4월 12일	
						2025년 4월 11일	
						2025년 4월 13일	
						2025년 4월 18일	

• 진행 과정 with Github

Organization : <https://github.com/DE-Project1>



02

주제 정의 및 필요성

- “Where2Eat”



국물요리?

비 오니까
따뜻한 국물이 땡긴다..
뭐라고 검색하지?

비 오는 날 밥 먹을 곳?

기념일이라 조용하고
특별한 분위기의 식당을 찾고 있어

- “Where2Eat”

이미 키워드 중심 맛집 추천 서비스는 포화상태



사용자는 단순한 키워드가 아닌,
“상황(context)”에 맞는 맞춤형 추천을 원한다.



나의 구체적인 상황에 대한 맛집을 알 수 있다면 어떨까?

각 음식점의 리뷰 데이터를 분석해서 해당 음식점에 방문하는 상황을 도출해보자 !

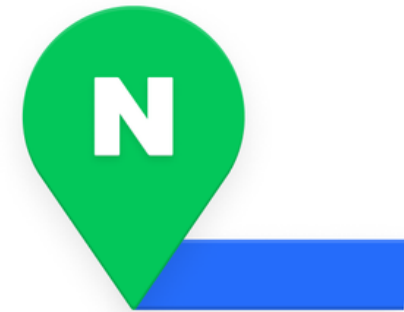
03

데이터 수집

• 크롤링 개요

[크롤링 대상 사이트]

- 네이버 지도에 등록된 음식점을 크롤링
- PC 네이버지도 검색 페이지, 모바일 네이버 플레이스 상세페이지



[크롤링 목표]

- “서울특별시 내 전체 구의 동별 맛집 정보 및 리뷰” 수집
- 서울 행정동 425개 > 동마다 25개↑ 음식점 > 음식점마다 리뷰 100개
- 총 100만건 이상의 음식점 리뷰 데이터 크롤링

• 크롤링 개요

[크롤링 도구: Playwright]



1. Chromium(크롬) 브라우저를 지원
2. Python 언어 지원
3. 네트워크 문제 상황에서 자동으로 연결 재시도
4. 동적 크롤러
 - 네이버 지도는 JavaScript 렌더링 이후 iframe 내부에 동적으로 로드되어, 정적 크롤러 (requests, BeautifulSoup 등)로는 접근이 불가
5. 비동기 크롤링 가능
 - 크롤링 시간을 획기적으로 단축

• 크롤링 개요

[크롤링 애플리케이션 개발 및 진행]

- Python + **Playwright v1.51.0** 사용
- 패키지 관리 도구로 **uv** 사용
- **html (Raw Data)**과 **json (음식점&리뷰 데이터)** 수집
- 크롤링 결과 파일 로컬에 저장 → **구글 드라이브**에 업로드
- 팀원 7명이 분담하여 진행
 - '구 단위'로 분배 / 인 당 3~4개의 구 담당
 - 인 당 1~2일 소요 (도합 10~14일 분량)

• 크롤링 음식점 조건

검색 키워드

‘지역구 + 동 + 맛집’으로 검색
ex) 마포구 연남동 맛집
→ 검색 결과 최적화 위함

업종 분류

네이버 스마트 플레이스 업종목록 기준
‘허용 카테고리 리스트’ 만들어서 사용
ex. 카페나 Bar 제외

```
ACCEPTED_CATEGORIES = [
    # 한식 계열
    "한식", "쌈밥", "보리밥", "비빔밥", "국밥", "찌개,전골", "곰탕,설렁탕", "김치찌개", "부대찌개", "청국장",
    "추어탕", "감자탕", "해장국", "갈비탕", "두부요리", "칼국수,만두", "전,빈대떡", "백반,가정식", "한식뷔페", "한정식",
    "국수", "냉면", "막국수", "삼계탕", "주꾸미요리", "순대,순댓국", "백숙,삼계탕", "치킨,닭강정", "생선구이",
    # 고기 계열
    "육류,고기요리", "달걀", "고기뷔페", "돼지고기구이", "족발,보쌈", "곱창,막창,양", "고기요리", "소고기구이", "닭요리",
    # 분식 계열
    "딤섬,중식만두", "중합분식", "김밥", "떡볶이", "라면", "만두", "주먹밥", "분식",
    # 일식 계열
    "일식당", "돈가스", "샤브샤브", "오니기리", "오므라이스", "초밥,롤", "우동,소바", "일본식라면",
    "일식튀김,꼬치", "카레", "일식", "이자카야", "참치회", "회전초밥",
```

리뷰 수

방문자 리뷰 수 140개 이상
공개된 영수증 리뷰 수 100개 이상
→ 신뢰 가능한 표본 확보 위함

별점

제공되는 경우 4.1점 이상
→ 맛집 확보 위함

• 크롤링 요소

[음식점 데이터]

- 장소 ID
- 상호명
- 분류 (한식, 중식 등)
- 주소
- 영업시간
- 제공 서비스
- 별점
- 방문자 리뷰 수
- 블로그 리뷰 수
- 배지



• 크롤링 요소

[리뷰 데이터]

- 장소 ID
- 작성자 닉네임
- 내용
- 작성 날짜
- 방문 상황 키워드
- 음식점 후기 키워드
- 작성자의 전체 리뷰 개수
- 해당 음식점 방문 횟수



• 크롤링 파이프라인

1. 크롤링 할 행정동 코드 확인 및 입력

`adm_dong_list.csv, target_adm_dong_codes.txt`

2. 크롤링 코드 돌리면 아래의 과정이 순차적으로 진행됨

`main.py`

a. 네이버 지도에 “OO구 OO동 맛집” 검색 → 조건에 맞는 장소 ID 수집

`crawler_pipeline.py, place_searcher.py`

b. 장소 ID로 상세페이지 접근 → 음식점 정보 및 리뷰 크롤링 진행

`place_data_collector.py`

c. 수집한 음식점 및 리뷰 데이터 json으로 저장

`save_data.py, json_data/place_info` 폴더, `json_data/reviews` 폴더

d. 음식점마다 Raw Data를 html로 저장

`save_data.py, raw_data` 폴더

• 크롤링 Challenges

레이지 로딩

Lazy Loading

: 사용자가 필요한 콘텐츠만 우선 로딩하는 방식

→ ‘동별 맛집’ 검색 시 검색 결과가 한번에 로딩되지 않음

→ 자체적으로 스크롤을 내릴 수 있도록 코드 수정

홈 탭

음식점의 홈 탭 페이지에서
영업시간 등 일부 정보가
표시되지 않음

→ 접혀있는 요소의 토글을
펼친 후 크롤링



크롤링 시간

시간이 너무 오래 걸림

→ Python의 asyncio 라이브러리와 Playwright의 async_playwright로 비동기 크롤링 도입

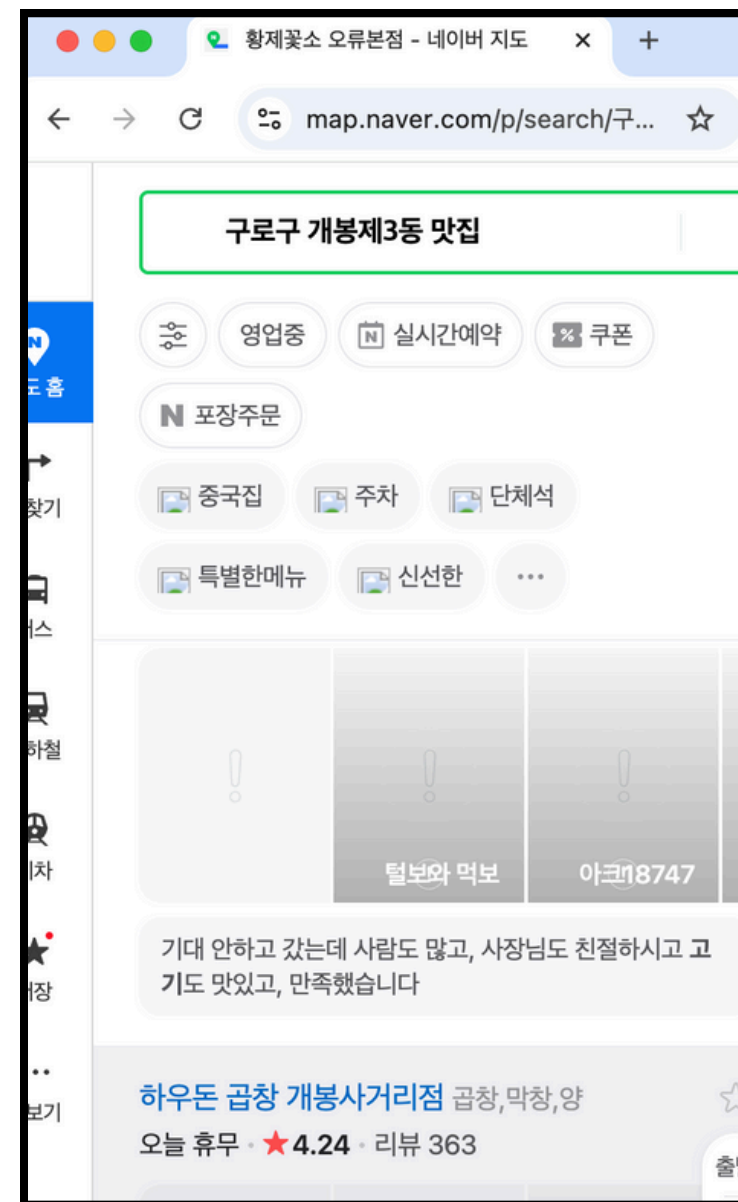
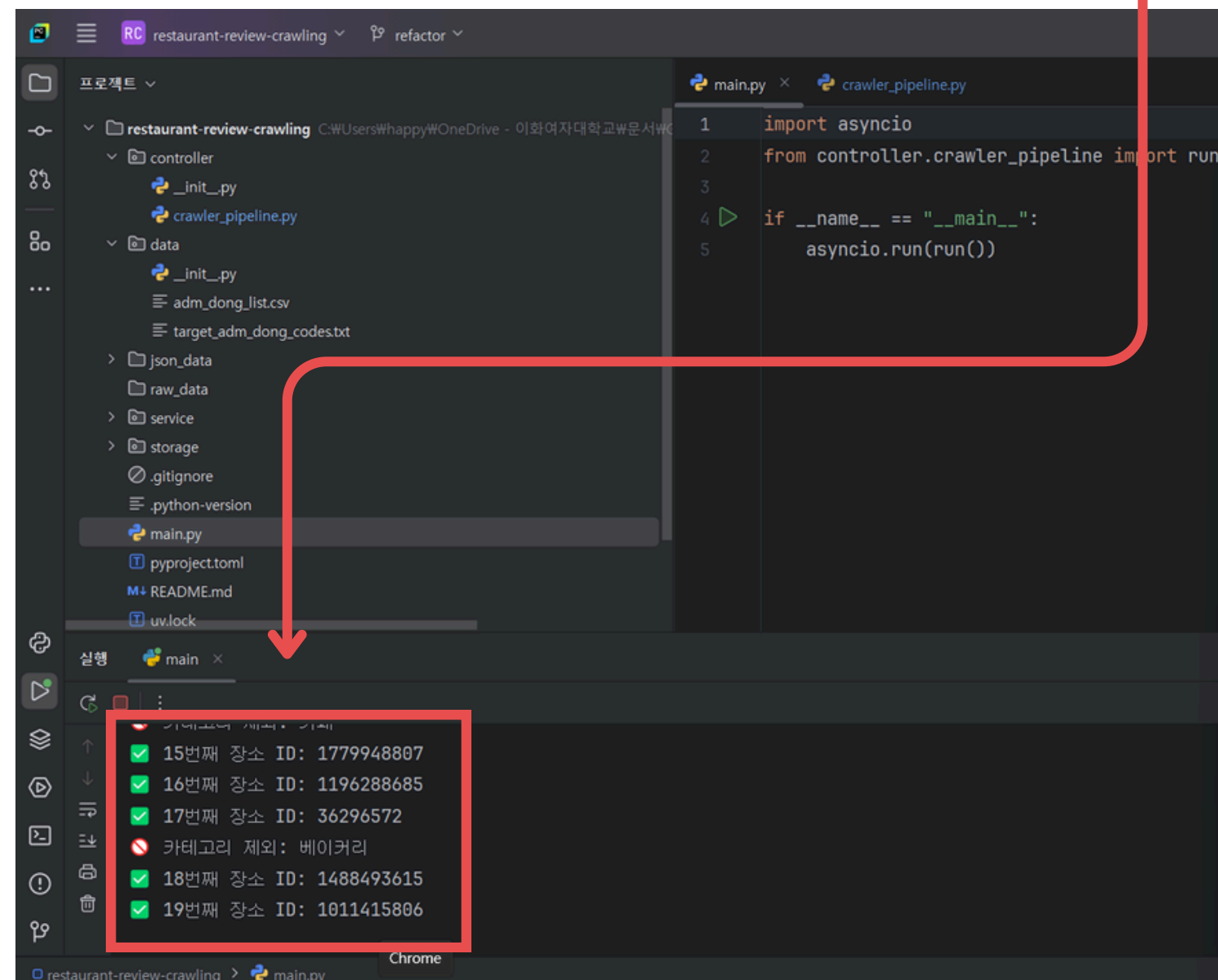
→ 여러개의 탭을 띄워 동시에 크롤링 진행, I/O 대기 시간을 겹쳐 처리해 속도 향상

• 크롤링 진행 모습

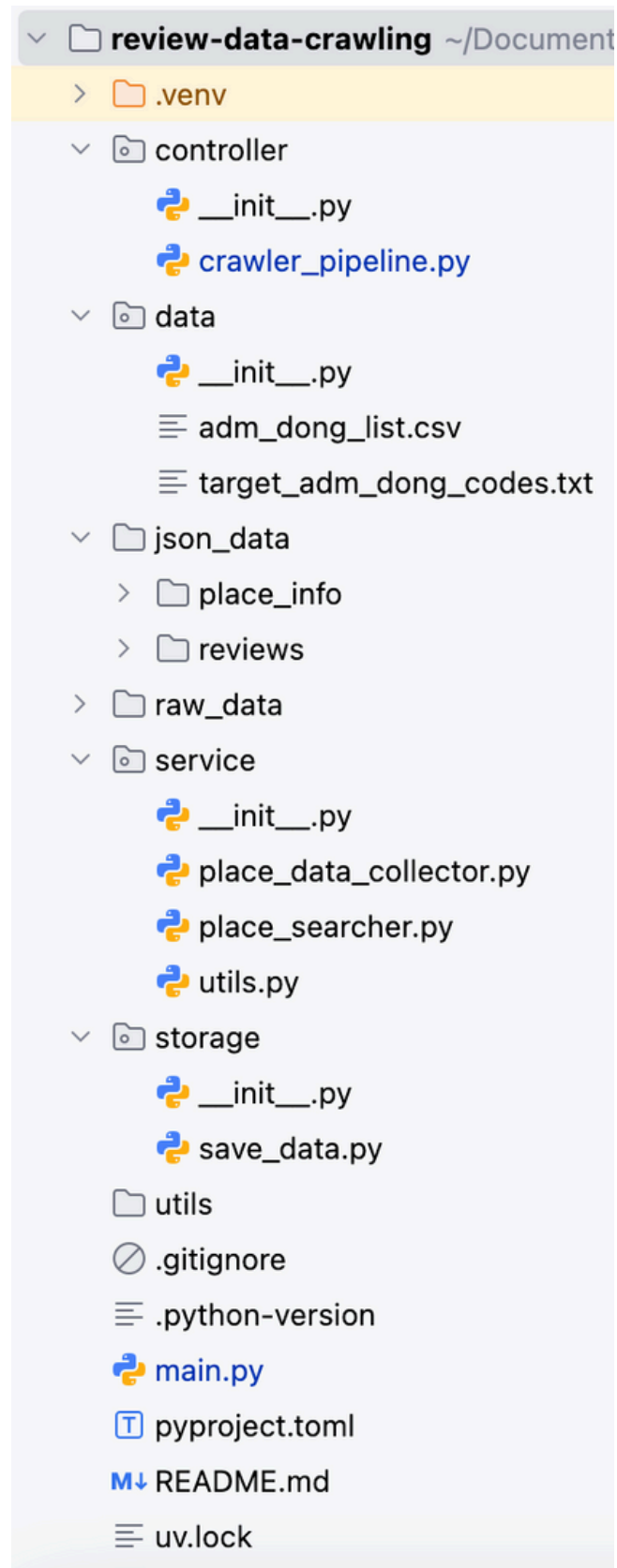
- main.py 실행 시 크롤링 화면
- 크롬창 뜨면서 크롤링 진행됨

1. 음식점 필터링, ID 수집
콘솔에서 필터링 확인 가능
조건에 맞는 장소 ID 수집

2. 비동기식으로 크롤링
→ 음식점, 리뷰, Raw Data 수집
→ 음식점 4곳 * 탭 3개씩 = 12개 탭



크롤링 주요 코드 설명



- 프로세스 순서대로 나열

• data/

- **adm_dong_list.csv**: 서울특별시 행정동코드, 시, 구, 동 정보가 작성된 파일
- **target_adm_dong_codes.txt**: 크롤링할 행정동코드를 입력하는 파일

	adm_dong_code	city	district	neighborhood
1	1111051500	서울특별시	종로구	청운효자동
2	1111053000	서울특별시	종로구	사직동
3	1111054000	서울특별시	종로구	삼청동
4	1111055000	서울특별시	종로구	부암동
5	1111056000	서울특별시	종로구	평창동
6	1111057000	서울특별시	종로구	무악동

1	1120054000
2	1120055000
3	1120061500
4	1120062000
5	1120064500
6	1120066000

• **main.py**: 크롤링 파이프라인 실행

• controller/

- **crawler_pipeline.py**: 전체 크롤링 파이프라인 코드 (타겟 행정동에 대해 맛집 검색과 개별 음식점 데이터 크롤링을 진행. 개별 음식점 데이터 크롤링은 4곳씩 비동기 크롤링)

```

1 import asyncio
2 from controller.crawler_pipeline import run
3
4 if __name__ == "__main__":
5     asyncio.run(run())
  
```

• service/

- **place_searcher.py**: 장소 검색 페이지에서 조건에 맞는 음식점만 크롤링
- **place_data_collector.py**: 네이버 플레이스에서 홈 탭과 정보 탭, 리뷰 탭 크롤링
- **utils.py**: 유용성을 위한 메소드들이 담긴 파일

• storage/**save_data.py**: json 파일 로딩/저장, html 파일로 저장하는 메소드가 작성된 파일

• json_data/

- **place_info/**: 동 단위로 음식점들의 크롤링된 장소 정보 데이터 json 파일이 저장되는 위치
- **reviews/**: 동 단위로 음식점들의 크롤링된 리뷰 데이터 json 파일이 저장되는 위치

• **raw_data/**: 개별 음식점 단위로 크롤링된 전체 정보 html 파일이 저장되는 위치

• 크롤링 코드 구현 설명

main.py > **run()**

crawling_pipeline.py

run() : 입력을 읽어 행정동별로 크롤링을 진행시키는 메소드

- 크롤링할 행정동코드 목록을 로드
(data/target_adm_dong_codes.txt 파일에 입력한)
- 행정동 정보가 들어있는 파일(data/adm_dong_list.csv)을 열어
각 행정동코드의 “구(district) 동(neighborhood)” 텍스트 추출
- 각 행정동코드(adm_dong_code)에 대해 다음을 진행

1. search_and_fetch_place_ids(“OO구 OO동”, MAX_PLACES)에서
장소ID리스트(place_ids) 수집

2. crawl_from_place_ids(place_ids, adm_dong_code)에서
개별 장소 데이터 크롤링 실행

```
import csv
import asyncio
import os
import time
from playwright.async_api import async_playwright
from service.place_searcher import search_and_fetch_place_ids
from service.place_data_collector import fetch_home_page_and_get_place_info, fetch_review_page_and_get_reviews, fetch_info_page
from service.utils import block_unnecessary_resources, log
from storage.save_data import save_place_info_json, save_reviews_json, save_place_raw_html, save_failed_places_json

RAW_DIR="raw_data"
TARGET_TXT_PATH="data/target_adm_dong_codes.txt" # 크롤링할 행정동코드
ADM_DONG_CSV_PATH="data/adm_dong_list.csv" # 행정동 csv 파일
MAX_PLACES=60

# 메인 실행 함수
async def run() -> None:
    # 파일을 입력으로 대상 행정동코드를 읽어옴
    with open(TARGET_TXT_PATH, 'r', encoding='utf-8-sig') as f:
        target_codes = set(line.strip() for line in f if line.strip())

    # 행정동코드로 구+동 정보 획득
    adm_dong_dict = {}
    with open(ADM_DONG_CSV_PATH, newline='', encoding='utf-8-sig') as csvfile:
        reader = csv.DictReader(csvfile)
        for row in reader:
            adm_dong_code = row["adm_dong_code"]
            if adm_dong_code in target_codes:
                # "구 동" 형태로 value 구성
                adm_dong_dict[adm_dong_code] = f"{row['district']} {row['neighborhood']}"

    for adm_dong_code in target_codes:
        global_start = time.time()
        # 크롤링 실행 (실패 후 중복 확인 로직 추가)
        try:
            place_ids = await search_and_fetch_place_ids(adm_dong_dict[adm_dong_code], MAX_PLACES) # 최대 수집 장소 수
            print(f"🚀 {adm_dong_code}: {len(place_ids)}개 수집 시작")
            await crawl_from_place_ids(list(place_ids), adm_dong_code)

        except Exception as e:
            print(f"❌ [ERROR] 크롤링 중 오류 발생 - {e}")

    global_elapsed = time.time() - global_start
    print(f"✅ 전체 소요시간: {global_elapsed:.1f} sec\n")
```


• 크롤링 코드 구현 설명

run() > 1. 장소ID리스트(place_ids) 수집

place_searcher.py

search_and_fetch_place_ids (district_neighborhood, max_places)

: 행정동 맛집을 네이버 지도에 검색하고, 결과에서 음식점 조건을 만족하는 장소들의 ID를 max_places개 찾는 메소드

- URL: https://map.naver.com/p/search/{district_neighborhood}%20맛집
- browser(Chromium), context, page 세팅
- utils.py에 정의한 block_unnecessary_resources 호출 → page의 이미지, 스타일시트, 폰트 요청을 차단해 불필요한 리소스 로딩 막고 크롤링 속도 개선
- 검색 결과는 동적 로딩됨: **iframe#searchIframe**
 - 해당 페이지 첫 진입 시 div#_pcmap_list_scroll_container 스크롤 다운해 전체 요소 로딩 후 다시 맨 위로 올라와 개별적 요소 접근
 - [parse_places_from_items](#)에서 음식점 조건에 부합하는지 확인, 부합하면 장소ID리스트(place_ids)에 추가
- 장소 검색 결과 페이지 이동:
 - a.eUTV2:has(span.place_blind:text("다음페이지"))로 버튼 찾아 클릭

```
# 장소 검색 및 ID 리스트 크롤링 함수
# 특정 지역에서 조건을 만족하는 장소 최대 max_places개까지 수집 (place_id만)
async def search_and_fetch_place_ids(district_neighborhood: str, max_places: int) -> List[str]:
    """ 2개의 사용 위치 ㄹ Seohyun Lee +1 """
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=False, args=["--window-size=400,800"])
        context = await browser.new_context(
            viewport={"width": 400, "height": 800},
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
            "AppleWebKit/537.36 (KHTML, like Gecko) Chrome/119.0.0.0 Safari/537.36"
        )
        page = await context.new_page()
        await block_unnecessary_resources(page)
        # 페이지 이동
        search_word = district_neighborhood + " 맛집"
        await page.goto(f"https://map.naver.com/p/search/{search_word}")
        await page.wait_for_timeout(2000) # 초기 로딩 대기
        # place_ids에 장소 ID 수집
        place_ids = []
        current_page = 1
        while current_page <= 5 and len(place_ids) < max_places: # 최대 페이지 번호 5; 장소 수 max_places 이하여야
            try:
                print(f"\n=== {current_page}페이지 크롤링 시작 ===")
                iframe_element = await page.wait_for_selector("iframe#searchIframe", timeout=5000)
                search_frame = await iframe_element.content_frame()
                scroll_container = await search_frame.wait_for_selector("div#_pcmap_list_scroll_container", timeout=10000)
                await scroll_until_no_more(scroll_container) # 스크롤 다옴으로 전체 아이템 로딩
                place_items = await search_frame.query_selector_all("li.UeZoS.rTjJo")
                print(f"✅ {current_page}페이지에서 {len(place_items)}개 장소 발견")
                # place_items에서 장소 new_ids 얻어 place_ids에 추가
                await parse_places_from_items(place_items, page, max_places, place_ids)
                if len(place_ids) >= max_places:
                    break
                # 다음 페이지 넘기기
                next_btn = search_frame.locator('a.eUTV2:has(span.place_blind:text("다음페이지"))').first
                if await next_btn.count() == 0:
                    print("❌ 다음페이지 버튼 못 찾음. 종료")
                    break
                # 비활성화 상태인지 확인
                if await next_btn.get_attribute("aria-disabled") == "true":
                    print("🚫 다음 페이지 버튼이 비활성화됨. 종료")
                    break
                # 클릭 후 대기
                current_page += 1
                await next_btn.click()
                await page.wait_for_timeout(1500)
            except Exception as e:
                print(f"[ERROR] 페이지 처리 중 오류: {e}")
                break
        await context.close()
        await browser.close()
        return place_ids
```

크롤링 코드 구현 설명

run() > 1. 장소ID리스트(place_ids) 수집

place_searcher.py

parse_places_from_items (items, page, max_places, place_ids)

: 장소가 음식점 조건에 맞다면 장소ID리스트에 추가하는 메소드

- **카테고리 검사:** span.KCMnt → ACCEPTED_CATEGORIES에 포함 확인
- **방문자 리뷰 수 검사:** span.h69bs → “리뷰” 텍스트에서 숫자 파싱 → 140 이상
- **별점 검사:** span.h69bs.orXYY → 평점 텍스트 존재하는 경우 실수 파싱 → 4.1 이상
- **장소 ID 추출:** div.place_bluelink 클릭 → 페이지 이동 됨, page.url 정규식 /place/(\d+) 매칭 → place_id 획득

```
# 수집 대상인 음식점 카테고리 리스트
ACCEPTED_CATEGORIES = [
    # 한식 계열
    "한식", "쌈밥", "보리밥", "비빔밥", "국밥", "짜개, 전골", "곰탕, 설렁탕",
    "추어탕", "감자탕", "해장국", "갈비탕", "두부요리", "칼국수, 만두", "국수",
    "냉면", "막국수", "삼계탕", "주꾸미요리", "순대, 순댓국", "백종원",
    # 고기 계열
    "육류, 고기요리", "닭발", "고기뭇밥", "돼지고기구이", "죽발, 보쌈", "곱창",
    # 분식 계열
    "떡볶이, 떡볶이", "김밥", "떡볶이", "라면", "만두", "주먹밥",
    # 일식 계열
    "일식", "돈가스", "샤부샤부", "오니기리", "오므라이스", "초밥, 롤",
    "일식튀김, 고치", "카레", "일식", "이자카야", "참치회", "회전초밥",
    # 중식 계열
    "중식", "양꼬치", "마라탕", "계요리", "담심", "중식", "중화요리",
    # 양식 계열
    "브런치", "스테이크, 힙", "피자", "파스타", "패밀리레스토랑",
    "경양식", "스파게티", "브런치카페", "양식", "독일음식", "스파게티, 파스타",
    # 세계 음식
    "이탈리아음식", "스페인음식", "프랑스음식", "터키음식", "아프리카음식",
    "멕시코, 남미음식", "베트남음식", "태국음식", "인도음식", "아시아음식",
    "멕시코, 브라질", "퓨전음식",
    # 주점
    "요리주점", "술집", "포장마차", "바(BAR)", "와인바", "와인", "맥주",
    # 그 외 음식점
    "푸드트럭", "푸드코트", "토스트", "햄버거", "죽", "도시락", "샐러드",
    "다이어트, 샐러드", "채식, 샐러드뷔페", "얼밥", "찰판요리", "치킨", "패스트푸드",
    "아식", "사찰, 영양탕", "사찰음식", "기타식당", "황토음식", "이북음식",
    "해물, 생선요리", "일식, 초밥뷔페", "해산물뷔페", "생선회", "도시락, 김밥"
]
```

```
MIN_REVIEW_COUNT = 140 # 방문자 리뷰
MIN_RATING = 4.1 # 최소 별점 기준
```

```
async def parse_places_from_items(items, page, max_places: int, place_ids):
    for item in items:
        if len(place_ids) >= max_places:
            break
        try:
            # 카테고리
            category_el = await item.query_selector("span.KCMnt")
            category = await category_el.text_content() if category_el else "N/A"
            if category not in ACCEPTED_CATEGORIES:
                print(f"❌ 카테고리 제외: {category}")
                continue
            # 방문자 리뷰
            review_el = await item.query_selector_all("span.h69bs")
            review_count = 0
            for span in review_el:
                text = await span.inner_text()
                if "리뷰" in text:
                    digits = re.search(pattern=r"(\d+)", text)
                    if digits:
                        review_count = int(digits.group(1))
                    break
            if review_count < MIN_REVIEW_COUNT:
                print(f"❌ 리뷰 수 부족: {review_count}")
                continue
            # 별점
            rating_el = await item.query_selector("span.h69bs.orXYY")
            rating_text = await rating_el.text_content() if rating_el else None
            if rating_text:
                match_rating = re.search(pattern=r"(\d+)", rating_text)
                rating = float(match_rating.group()) if match_rating else 0.0
                if rating < MIN_RATING:
                    print(f"❌ 별점 낮음: {rating}")
                    continue
            # 상세 페이지 이동 후 ID 추출
            click_target = await item.query_selector("div.place_bluelink")
            if click_target:
                await click_target.click()
                await page.wait_for_timeout(1500)
                detail_url = page.url
                match = re.search(pattern=r'/place/(\d+)', detail_url)
                if match:
                    place_id = match.group(1)
                    print(f"✅ {len(place_ids) + 1}번째 장소 ID: {place_id}")
                    place_ids.append(place_id)
                else:
                    print(f"❌ place_id 추출 실패")
            except Exception as e:
                print(f"[ERROR] 항목 처리 중 오류: {e}")
                continue
```

• 크롤링 코드 구현 설명

run() > 2. 개별 장소 데이터 크롤링 수행

crawling_pipeline.py

crawl_from_place_ids (place_ids, adm_dong_code)

: 장소들을 최대 4개씩 동시에 크롤링하기 위한 메소드

1. **async_playwright**으로 비동기 크롤링 진행

2. browser, context 생성 (viewport, User-Agent 설정)

3. 장소 ID 4개씩을 배치로 묶어 작업

- 4개 각각의 place_id로 **scrape_place_details(context, place_id, adm_dong_code)** 코루틴 객체를 생성해 tasks 리스트에 추가 (예약된 작업으로 보관)
- **await asyncio.gather(*tasks, return_exceptions=True)** 호출 시 코루틴 4개가 비동기 실행
→ 장소 4곳의 상세페이지 크롤링 메소드가 동시 실행

```
# place_ids로부터 음식점을 4개씩 배치 크롤링
async def crawl_from_place_ids(place_ids: list[str], adm_dong_code): 1개의 사용 위치 ㄹ Seohyun Lee
    results = []
    async with async_playwright() as p:
        place_ids = list(place_ids)

        browser = await p.chromium.launch(headless=False)
        context = await browser.new_context(
            viewport={"width": 400, "height": 800},
            user_agent="Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like G
        )

        # 4개씩 처리
        for i in range(0, len(place_ids), 4):
            batch = place_ids[i:i + 4]

            tasks = []
            for place_id in batch:
                output_path = f"{RAW_DIR}/adc_{adm_dong_code}_place_rawdata_{place_id}.html"
                if os.path.exists(output_path):
                    print(f"이미 파일 있음 → 스킵 (PlaceID: {place_id})")
                    continue
                tasks.append(scrape_place_details(context, place_id, adm_dong_code))

            # 동시에 실행
            batch_results = await asyncio.gather(*tasks, return_exceptions=True)

            # 결과 저장
            for result in batch_results:
                if isinstance(result, dict):
                    results.append(result)
            await context.close()
            await browser.close()

    return results
```

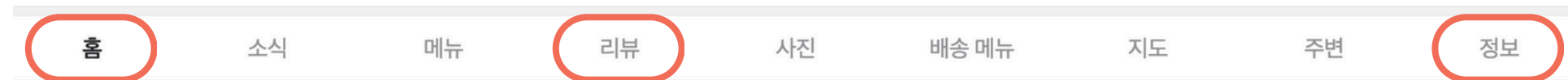

• 크롤링 코드 구현 설명

run() > 2. 개별 장소 데이터 크롤링 수행

crawling_pipeline.py

scrape_place_details (context, place_id, adm_dong_code)

: 한 장소의 네이버 플레이스 상세페이지 데이터를 수집을 위한 메소드 -
홈 탭, 리뷰 탭, 정보 탭 동시에 크롤링 실행



1. **현 context에서 page 3개 생성**
2. 각 page 대해 block_unnecessary_resources 호출 → 불필요한 리소스 로딩 차단
3. **asyncio.gather()로 3개 페이지를 비동기 크롤링 실행:**
 - fetch_home_page_and_get_place_info(home_page, ...)
 - fetch_review_page_and_get_reviews(review_page, ...)
 - fetch_info_page(info_page, ...)
4. **HTML 및 JSON 저장 메소드 호출:**
 - page.content() 로 raw HTML 추출
 - save_place_info_json, save_reviews_json, save_place_raw_html
5. 완료 시 page 3개 모두 닫음

```
# 식당 1개 크롤링
async def scrape_place_details(context, place_id: str, adm_dong_code: int): 1개의 사용 위치  ⚡ Seohyun Lee
    log("START", place_id)
    start_time = time.time()
    html_data = {"place_id": place_id, "home_html": None, "info_html": None, "reviews_html": None}

    # 탭 열기
    home_page = await context.new_page()
    info_page = await context.new_page()
    review_page = await context.new_page()

    # 리소스 차단
    await block_unnecessary_resources(home_page)
    await block_unnecessary_resources(info_page)
    await block_unnecessary_resources(review_page)

    # 세 가지 작업을 병렬로 실행. 하단과도 예외 발생 시 전체가 예외를 던짐
    results = await asyncio.gather(
        fetch_home_page_and_get_place_info(home_page, place_id, adm_dong_code),
        fetch_review_page_and_get_reviews(review_page, place_id),
        fetch_info_page(info_page, place_id),
        return_exceptions=True
    )
    place_info, reviews, info_html = results
    if isinstance(place_info, Exception) or isinstance(info_html, Exception) or isinstance(reviews, Exception):
        # 하단과도 실패 시 함수 전체 종료
        print(f"❌ {place_id} - 수집 불충분 → 저장 생략")
        await home_page.close()
        await info_page.close()
        await review_page.close()
        return # 함수 전체 종료

    try:
        html_data["home_html"] = await home_page.content()
        html_data["info_html"] = info_html
        html_data["reviews_html"] = await review_page.content()

        save_place_info_json(place_info, adm_dong_code)
        save_reviews_json(reviews, adm_dong_code)
        save_place_raw_html(place_id, adm_dong_code, html_data)

        elapsed = time.time() - start_time
        log("SAVED", place_id, extra=f"{elapsed:.1f} sec")
    except Exception as e:
        save_failed_places_json(place_id, adm_dong_code)
        log("ERROR", place_id, extra=str(e))
    finally: # 페이지 닫기
        await home_page.close()
        await info_page.close()
        await review_page.close()
```

• 크롤링 코드 구현 설명

run() > 2. 개별 장소 데이터 크롤링 수행

place_data_collector.py

fetch_home_page_and_get_place_info (page, place_id, adm_dong_code)

: 홈 탭을 크롤링하는 메소드

- URL: **https://m.place.naver.com/restaurant/{place_id}/home**
- 상호명: span.GHAhO, 카테고리: span.lnJFt, 주소: span.LDgIH
- 영업시간: a.gKP9i[aria-expanded="false"] 토크 클릭해 전체 영업시간 로드 (parse_opening_hours(page)로 파싱)
- 네이버 평점: span.PXMot.LXlwF → 평점 텍스트에서 숫자 파싱
- 방문자·블로그 리뷰 수: a[href*="/review/visitor"], a[href*="/review/ugc"]
- 제공 서비스: div.xPvPE, 배지: div.XtBbS
- 크롤링 시각 기록: datetime.now()

fetch_info_page (page, place_id)

: 정보 탭을 크롤링하는 메소드

- URL: **https://m.place.naver.com/restaurant/{place_id}/information**
- 로딩 대기: page.wait_for_load_state('domcontentloaded')
- 페이지 바닥까지 스크롤을 반복 실행해 모든 콘텐츠를 렌더링한 뒤 최종 HTML을 꺼내어 반환

```
import asyncio
import re
from datetime import datetime
import random
from playwright.async_api import Page
from service.utils import safe_page_content

# 장소 상세 정보 수집
# 홈 페이지로 이동
async def fetch_home_page_and_get_place_info(page, place_id, adm_dong_code):
    url = f"https://m.place.naver.com/restaurant/{place_id}/home"
    await page.goto(url)
    await page.wait_for_timeout(1000)
    info = {}

    try:
        # 상호명, 카테고리, 주소
        name_el = await page.query_selector('span.GHAhO')
        category_el = await page.query_selector('span.lnJFt')
        address_el = await page.query_selector('span.LDgIH')
        await page.wait_for_timeout(random.randint(1000, 1500))

        # 영업시간 클릭해 펼쳐기
        toggle_el = await page.query_selector('a.gKP9i[aria-expanded="false"]')
        if toggle_el:
            await toggle_el.click()
            await page.wait_for_timeout(random.randint(800, 1200)) # 약간 대기

        # 서비스
        service_el = await page.query_selector('div.xPvPE')

        # 평점
        rating_el = await page.query_selector('span.PXMot.LXlwF')
        rating_text = await rating_el.text_content() if rating_el else None
        naver_rating = re.search(pattern=r"(\d)+", rating_text).group() if rating_text else "N/A"

        # 방문자 리뷰 수, 블로그 리뷰 수
        visitor_review_el = await page.query_selector('a[href*="/review/visitor"]')
        blog_review_el = await page.query_selector('a[href*="/review/ugc"]')
        visitor_review_text = await visitor_review_el.text_content() if visitor_review_el else ""
        blog_review_text = await blog_review_el.text_content() if blog_review_el else ""
        visitor_review_count = int(re.sub(pattern=r"(\d)+", repl=" ", visitor_review_text)) if visitor_review_text else 0
        blog_review_count = int(re.sub(pattern=r"(\d)+", repl=" ", blog_review_text)) if blog_review_text else 0
        if isinstance(blog_review_count, tuple):
            blog_review_count = blog_review_count[0]

        # 배지
        badge_els = await page.query_selector_all('div.XtBbS')
        badges = []
        for el in badge_els:
            text = await el.text_content()
            if text:
                badges.append(text.strip())

        # 수집 시각
        crawled_at = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        info["place_id"] = place_id
        info["name"] = await name_el.text_content() if name_el else "N/A"
        info["category"] = await category_el.text_content() if category_el else "N/A"
        info["address"] = await address_el.text_content() if address_el else "N/A"
        info["opening_hours"] = await parse_opening_hours(page)
        info["services"] = await service_el.text_content() if service_el else "N/A"
        info["naver_rating"] = naver_rating
        info["visitor_review_count"] = visitor_review_count
        info["blog_review_count"] = blog_review_count
        info["badges"] = ", ".join(badges) if badges else "N/A"
        info["crawled_at"] = crawled_at
        info["adm_dong_code"] = adm_dong_code
    except Exception as e:
        print(f"[{place_id}] Error crawling place info: {e}")
    return info
```

```
async def fetch_info_page(page: Page, place_id: str):
    url = f"https://m.place.naver.com/restaurant/{place_id}/information"
    await page.goto(url, timeout=15000)
    await page.wait_for_load_state(state='domcontentloaded', timeout=15000)
    for _ in range(6):
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight)")
        await asyncio.sleep(0.5)
    return await safe_page_content(page, timeout=10)
```


• 크롤링 코드 구현 설명

run() > 2. 개별 장소 데이터 크롤링 수행

place_data_collector.py

fetch_review_page_and_get_reviews (page, place_id)

: 리뷰 탭을 크롤링하는 메소드

- URL: https://m.place.naver.com/restaurant/{place_id}/review
- 공개된 영수증 리뷰 수 100개 이상인지 검사: 미달 시 크롤링 종료(저장X)
 - div.place_section_header_title → <em class="place_section_count"> 파싱
- 리뷰 각 항목: li.place_apply_pui
 - 100개 로딩 위해 더보기 버튼 클릭 필요: a.fvwqf 클릭 + 스크롤 다운
- 리뷰별 데이터
 - 닉네임: div.pui__JiVbY3 span.pui__NMI-Dp
 - 내용: div.pui__vn15t2 a
 - 날짜: div.pui__QKE5Pr > span.pui__gfuUIT > span.pui__blind 파싱
 - 방문 상황 키워드: a.pui__uqSlGI > span.pui__V8F9nN (여러 개)
 - 리뷰 키워드: div.pui__HLNvmI > span.pui__jhpEyP (여러 개)
 - 리뷰 수: div.pui__RuLAax > span:nth-child(1)
 - 방문 차수: div.pui__QKE5Pr > span.pui__gfuUIT:nth-child(2)

```
# 리뷰 크롤링
async def fetch_review_page_and_get_reviews(page, place_id): 2개의 사용 위치   SeoHyun Lee
    url = f"https://m.place.naver.com/restaurant/{place_id}/review"
    await page.goto(url)
    await page.wait_for_timeout(5000)

    MAX_REVIEWS = 100
    review_header = await page.query_selector('div.place_section_header_title')
    if review_header:
        review_html = await review_header.inner_html()
        match = re.search(pattern=r'<em class="place_section_count">(\d*)</em>', review_html)
        if match:
            total_reviews = int(match.group(1))
            if total_reviews < MAX_REVIEWS:
                print(f"❌ 리뷰 수 부족: {total_reviews}개 → 수집 제외")
                raise ValueError("리뷰 수 부족")

    # 스크롤/더보기 버튼 통해 최대한 리뷰 로딩 (최대 MAX_REVIEWS개)
    for _ in range(10): # 더보기 버튼 클릭 최대 횟수
        review_items = await page.query_selector_all("li.place_apply_pui")
        if len(review_items) >= MAX_REVIEWS:
            break

        await page.mouse.wheel(0, 5000)
        await page.wait_for_timeout(500)

    prev_count = len(review_items)
    more_btn = await page.query_selector("a.fvwqf") # 더보기 버튼
    if more_btn:
        try:
            await more_btn.click()
            await page.wait_for_timeout(2000)

            # 더보기 클릭 후 리뷰 수가 늘어났는지 확인
            for _ in range(2): # 최대 2회 재시도
                new_items = await page.query_selector_all("li.place_apply_pui")
                if len(new_items) > prev_count:
                    break
                await page.wait_for_timeout(60000) # 1분 대기
            except Exception as e:
                print(f"⚠️ 더보기 클릭 실패: {e}")
                break
        else:
            break
    await page.wait_for_timeout(3000)

    review_items = await page.query_selector_all("li.place_apply_pui")

    review_count = len(review_items)
    if review_count < MAX_REVIEWS:
        print(f"리뷰 수 부족({review_count}개), 크롤링 생략")
        return ValueError("리뷰 수 부족")
    else:
        print(f"리뷰 수집 대상: {review_count}개")
```

```
result = []
for r in review_items[:MAX_REVIEWS]:
    try:
        nickname_el = await r.query_selector("div.pui__JiVbY3 span.pui__uslU0d span.pui__NMI-Dp")
        content_el = await r.query_selector("div.pui__vn15t2 a")

        nickname = await nickname_el.text_content() if nickname_el else "N/A"
        content = await content_el.text_content() if content_el else "N/A"

        # 날짜 element를 모두 찾기
        date_els = await r.query_selector_all("div.pui__QKE5Pr > span.pui__gfuUIT > span.pui__blind")
        date_raw = await date_els[1].text_content() if len(date_els) > 1 else "N/A"

        # 날짜 파싱
        date = "N/A"
        if date_raw != "N/A":
            m = re.search(pattern=r'(\d{4})년 (\d{1,2})월 (\d{1,2})일', date_raw)
            if m:
                y, mth, d = m.groups()
                date = f"{int(y):04d}-{int(mth):02d}-{int(d):02d}"

        # 방문 상황 키워드(점심, 데이트, 바운팅 등)
        situation_els = await r.query_selector_all("a.pui__uqSlGI > span.pui__V8F9nN")
        situations = [await el.text_content() for el in situation_els]
        situations_str = ", ".join([s.strip() for s in situations])

        # 리뷰 키워드(맛, 분위기 등)
        keyword_els = await r.query_selector_all("div.pui__HLNvmI > span.pui__jhpEyP")
        keywords = [await el.text_content() for el in keyword_els]
        keywords_str = ", ".join([k.strip() for k in keywords])

        # 리뷰 개수 및 방문 차수 추출
        review_count_el = await r.query_selector("div.pui__RuLAax > span:nth-child(1)")
        visit_count_el = await r.query_selector("div.pui__QKE5Pr > span.pui__gfuUIT:nth-child(2)")
        review_count = await review_count_el.text_content() if review_count_el else "N/A"
        visit_count = await visit_count_el.text_content() if visit_count_el else "N/A"

        # 리뷰 수: "리뷰 14" → 14
        review_count = re.sub(pattern=r"^\d+", repl="", review_count)
        # 방문 차수: "1번째 방문" → 1
        visit_count = re.sub(pattern=r"^\d+", repl="", visit_count)

        review_data = {
            "place_id": place_id,
            "nickname": nickname.strip(),
            "content": content.strip(),
            "date": date.strip(),
            "situations": situations_str,
            "keywords": keywords_str,
            "review_count": review_count,
            "visit_count": visit_count
        }
        print(f"수집 완료: {review_data}")
        result.append(review_data)
    except Exception as e:
        print(f"[{place_id}] Error parsing review: {e}")

return result
```

• 크롤링 코드 구현 설명

2. 개별 장소 데이터 크롤링 수행 > 저장

save_data.py

1. save_place_info_json(place_info, adm_dong_code)

- 행정동 별로 장소(음식점) 데이터를 json에 누적해서 저장

2. save_reviews_json(reviews, adm_dong_code)

- 행정동 별로 리뷰 데이터를 json에 누적해서 저장

3. save_failed_places_json(place_id, adm_dong_code)

- 실패한 장소 ID를 json으로 따로 저장

4. save_place_raw_html(place_id, adm_dong_code, html_data)

- 음식점 별로 Raw Data를 html로 저장

```

save_data.py
1  import json
2  import os
3  from typing import List, Dict
4
5  PLACE_INFO_DIR = "json_data/place_info"
6  REVIEWS_DIR = "json_data/reviews"
7  FAILED_PLACES_DIR = "json_data/failed_places"
8  RAW_DATA_DIR = "raw_data"
9
10 def _load_json_list(path: str) -> List[Dict]:
11     """JSON 파일을 불러오거나 없으면 빈 리스트를 반환"""
12     if os.path.exists(path):
13         with open(path, "r", encoding="utf-8-sig") as f:
14             return json.load(f)
15     return []
16
17 def _save_json_list(path: str, data: List[Dict]) -> None:
18     """리스트 데이터를 JSON으로 저장"""
19     os.makedirs(os.path.dirname(path), exist_ok=True)
20     with open(path, "w", encoding="utf-8-sig") as f:
21         json.dump(data, f, ensure_ascii=False, indent=2)
22
23 def save_place_info_json(place_info: Dict, adm_dong_code: int) -> None:
24     """장소 정보를 JSON으로 저장"""
25     os.makedirs(PLACE_INFO_DIR, exist_ok=True)
26     path = os.path.join(PLACE_INFO_DIR, f"place_info_{adm_dong_code}.json")
27     data = _load_json_list(path)
28     data.append(place_info)
29     _save_json_list(path, data)
30
31 def save_reviews_json(reviews: List[Dict], adm_dong_code: int) -> None:
32     """리뷰 정보를 JSON으로 저장"""
33     if not reviews:
34         return
35     os.makedirs(REVIEWS_DIR, exist_ok=True)
36     path = os.path.join(REVIEWS_DIR, f"reviews_{adm_dong_code}.json")
37     data = _load_json_list(path)
38     data.extend(reviews) # 리뷰는 리스트라 extend로 추가
39     _save_json_list(path, data)
40
41 def save_failed_places_json(place_id: str, adm_dong_code: int) -> None:
42     """실패한 장소 정보를 JSON으로 저장"""
43     os.makedirs(FAILED_PLACES_DIR, exist_ok=True)
44     path = os.path.join(FAILED_PLACES_DIR, f"failed_places_{adm_dong_code}.json")
45     data = _load_json_list(path)
46     data.append({"adm_dong_code": adm_dong_code, "place_id": place_id})
47     _save_json_list(path, data)
48
49 def save_place_raw_html(place_id: str, adm_dong_code: int, html_data: dict) -> None:
50     """HTML 데이터를 저장 (이미 있으면 저장하지 않음)"""
51     os.makedirs(RAW_DATA_DIR, exist_ok=True)
52     html_path = os.path.join(RAW_DATA_DIR, f"adc_{adm_dong_code}_place_rawdata_{place_id}.html")
53     # 이미 HTML 저장됨 → 스킵
54     if os.path.exists(html_path):
55         return
56     try:
57         with open(html_path, "w", encoding="utf-8-sig") as f:
58             f.write("===== HOME =====\n")
59             f.write(html_data.get("home_html") or "")
60             f.write("\n\n===== INFO =====\n")
61             f.write(html_data.get("info_html") or "")
62             f.write("\n\n===== REVIEWS =====\n")
63             f.write(html_data.get("reviews_html") or "")
64     except Exception as e:
65         print(f"[ERROR] HTML 저장 실패 (PlaceID: {place_id}) - {e}")

```

● 크롤링 결과

[음식점&리뷰 JSON]

```

1 {
2   "place_id": "13005867",
3   "name": "은성보쌈",
4   "category": "족발,보쌈",
5   "address": "서울 성동구 독서당로 297-6",
6   "opening_hours": "월 10:30 - 22:20 화 10:30 - 22:20 수 정기휴무 (매주 수요일) 목 10:30 - 22:20 금 10:30 - 22:20 토 10:30 - 22:20",
7   "services": "포장",
8   "naver_rating": "4.5",
9   "visitor_review_count": 1,
10  "blog_review_count": 0,
11  "badges": "맛있는",
12  "crawled_at": "2025-04-21",
13  "adm_dong_code": "14",
14 },
15 {
16   "place_id": "105",
17   "name": "채원김치담",
18   "category": "김치",
19   "address": "서울",
20   "opening_hours": "13",
21   "services": "포장",
22   "naver_rating": "4.5",
23   "visitor_review_count": 16,
24   "blog_review_count": 17,
25   "badges": "골목한",
26   "crawled_at": "2025-04-21",
27   "adm_dong_code": "20",
28 },
29 {
30   "place_id": "117",
31   "name": "신금호순물",
32   "category": "육류",
33   "address": "서울",
34   "opening_hours": "26",
35   "services": "예약",
36   "naver_rating": "4.5",
37   "visitor_review_count": 29,
38   "blog_review_count": 30,
39   "badges": "맛있는",
40   "crawled_at": "2025-04-21",
41   "adm_dong_code": "33",
42 },
43 {
44   "place_id": "145",
45   "name": "한남동김치",
46   "category": "김치",
47   "address": "서울",
48   "opening_hours": "40",
49   "services": "단체",
50   "naver_rating": "4.5",
51   "visitor_review_count": 42,
52   "blog_review_count": 43,
53   "badges": "맛있는",
54   "crawled_at": "2025-04-21",
55   "adm_dong_code": "47",
56 },
57 {
58   "place_id": "13005867",
59   "nickname": "새초롬이51",
60   "content": "금남시장 내의 맛집으로 소문났다해서 가 봤어요~고기가 부드럽고, 같이 나오는 김치가 엄청 맛있네요. 굴전도 넘 맛있어요. 말치 국수는 국물",
61   "date": "2025-04-21",
62   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친척 형제",
63   "keywords": "음식이 맛있어요",
64   "review_count": "36",
65   "visit_count": "1"
66 },
67 {
68   "place_id": "13005867",
69   "nickname": "인생은 짧고 맛있는 건 많다",
70   "content": "나혼산 보고 갔는데 맛집 맞네요, 아들이들한 보쌈에 한접시 푸짐하게 나온 맛갈스런 김치 조합은 역시나.. 삼겹보쌈이 제일 부드럽고 맛남.",
71   "date": "2025-04-18",
72   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 가족모임, 부모님, 친척 형제",
73   "keywords": "음식이 맛있어요",
74   "review_count": "896",
75   "visit_count": "1"
76 },
77 {
78   "place_id": "13005867",
79   "nickname": "수달조아",
80   "content": "친파 오랜만에 방문했는데 맛있는 김치와 보쌈이 어찌나 맛있어서 좋았어요~ 삼겹살 들어간 보쌈보다는 기름기 적은 은성 보쌈이 더 담백해서 추천하",
81   "date": "2025-04-03",
82   "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 친목, 친구",
83   "keywords": "음식이 맛있어요",
84   "review_count": "52",
85   "visit_count": "1"
86 },
87 {
88   "place_id": "13005867",
89   "nickname": "버섯마왕JYP",
90   "content": "맛나맛나~",
91   "date": "2025-03-30",
92   "situations": "점심에 방문, 예약 없이 이용, 대기 시간 바로 입장, 데이트, 연인-배우자",
93   "keywords": "음식이 맛있어요",
94   "review_count": "1404",
95   "visit_count": "1"
96 },
97 {
98   "place_id": "13005867",
99   "nickname": "syo***",
100  "content": "굿",
101  "date": "2025-03-29",
102  "situations": "저녁에 방문, 예약 없이 이용, 대기 시간 바로 입장, 일상, 연인-배우자",
103  "keywords": "잠내가 적어요",
104  "review_count": "1087",
105  "visit_count": "1"
106 }
107 }

```

[Raw Data HTML]

[illegible]

• 크롤링 결과

[구글 드라이브에 저장]

- 음식점 & 리뷰 데이터 - JSON
 - 행정동코드 단위로 저장 (712.4MB)
 - place_info 파일: 425개
 - review 파일: 425개
- Raw Data - HTML
 - 음식점 단위로 저장 (20.560GB)
 - 파일: 17,157개



[place_info]

... > json-data > place_info	
유형	사람
수정 날짜	출처
이름	소유자
place_info_1111051500.json	나
place_info_1111053000.json	나

[review]

... > json-data > reviews	
유형	사람
수정 날짜	출처
이름	소유자
reviews_1111051500.json	나
reviews_1111053000.json	나

[html]

내 드라이브 > crawling > html	
유형	사람
수정 날짜	출처
이름	소유자
adc_1111051500_place_rawdata_11808181.html	나
adc_1111051500_place_rawdata_11828877.html	나

04

데이터 전처리 및 적재

• 데이터 로드

공용 Google Drive에 보관된 JSON 파일 로드해서 사용



crawling > json-data > place_info

...	>	json-data	>	place_info	▼	🔍
유형	▼	사람	▼	수정 날짜	▼	출처
이름	↓			소유자		
📄		place_info_1174070000.json	👤	나		
📄		place_info_1174069000.json	👤	나		
📄		place_info_1174068500.json	👤	나		
📄		place_info_1174066000.json	👤	나		
📄		place_info_1174065000.json	👤	나		
📄		place_info_1174064000.json	👤	나		
📄		place_info_1174062000.json	👤	나		
📄		place_info_1174061000.json	👤	나		

425개의 JSON파일

17,157개의 document

crawling > json-data > reviews

...	>	json-data	>	reviews	▼	🔍
유형	▼	사람	▼	수정 날짜	▼	출처
이름	↓			소유자		
📄		reviews_1174070000.json	👤	나		
📄		reviews_1174069000.json	👤	나		
📄		reviews_1174068500.json	👤	나		
📄		reviews_1174066000.json	👤	나		
📄		reviews_1174065000.json	👤	나		
📄		reviews_1174064000.json	👤	나		
📄		reviews_1174062000.json	👤	나		
📄		reviews_1174061000.json	👤	나		

425개의 JSON파일

1,726,634개의 document

용량상 mongodb atlas(무료 한도 : 500MB)에 바로 업로드 하지 못하고, 전처리 후 올림.

• 데이터 전처리 파이프라인

- | | |
|------------------------------------|--|
| 1. 장소 중복 제거 및 리뷰 정리 : | 중복된 place_id 제거 및 연관된 리뷰 정리 |
| <code>deduplicate_places.py</code> | |
| 2. 필요한 컬럼만 필터링 : | 장소 및 리뷰 데이터에서 필요한 컬럼만 추출 |
| <code>filter_columns.py</code> | |
| 3. 리뷰 텍스트 클렌징 : | 특수문자, 이모지, NaN 제거 등 텍스트 정제 |
| <code>clean_text.py</code> | |
| 4. 명사 추출 및 불용어 제거 : | 리뷰에서 명사만 추출하고 불용어 제거 |
| <code>extract_nouns.py</code> | |
| 5. 유효한 리뷰 필터링 : | 모든 과정 후, 너무 짧거나 의미 없는 리뷰 제거하여 유효 리뷰만 남김. (단어수 5개 이상) |
| <code>validate_review.py</code> | |

• 데이터 전처리 파이프라인

```
def run_pipeline(region_csv_path: str): 1 usage  Kaeun Han
    try:
        logger.info("🔥 Step 1: Google Drive에서 데이터 불러오는 중...")
        df_place_info_raw = fetch_place_info()
        df_reviews_raw = fetch_reviews()
        logger.debug(f"📦 Place info shape: {df_place_info_raw.shape}, Reviews shape: {df_reviews_raw.shape}")
    except Exception as e:
        logger.error(f"❌ Google Drive 데이터 불러오기 실패: {e}")
        return

    try:
        logger.info("🔍 Step 2: 중복 place_id 처리 및 리뷰 필터링 중...")
        df_place_dedup, df_reviews_dedup, _ = deduplicate_places(df_place_info_raw, df_reviews_raw)
        logger.debug(f"✅ Deduplicated places: {df_place_dedup.shape}, Deduplicated reviews: {df_reviews_dedup.shape}")
    except Exception as e:
        logger.error(f"❌ 중복 제거 실패: {e}")
        return

    try:
        logger.info("📁 Step 3: 컬럼 필터링...")
        df_place_filtered, df_reviews_filtered = filter_columns(df_place_dedup, df_reviews_dedup)
        logger.debug(f"📄 Filtered place columns: {df_place_filtered.columns.tolist()}")
        logger.debug(f"📄 Filtered review columns: {df_reviews_filtered.columns.tolist()}")
    except Exception as e:
        logger.error(f"❌ 컬럼 필터링 실패: {e}")
        return
```

Google Drive에서 JSON 파일 로드



장소 중복 제거 및 리뷰 정리



필요한 컬럼만 필터링

• 데이터 전처리 파이프라인

```
try:
    logger.info("🟢 Step 4: 리뷰 텍스트 클렌징...")
    logger.debug(f"🔍 클렌징 전 리뷰 수: {len(df_reviews_filtered)}")
    df_reviews_cleaned = clean_text(df_reviews_filtered)
    logger.debug(f"🔍 클렌징 후 리뷰 수: {len(df_reviews_cleaned)}")
    logger.debug(f"🔍 Cleaned review 예시: {df_reviews_cleaned['content'].iloc[0]}")
except Exception as e:
    logger.error(f"❌ 텍스트 클렌징 실패: {e}")
    return

try:
    logger.info("🧠 Step 5: 명사 추출 및 불용어 제거...")
    logger.debug(f"🔍 명사 추출 전 리뷰 수: {len(df_reviews_cleaned)}")
    df_reviews_nouns = extract_nouns_from_reviews(df_reviews_cleaned)
    logger.debug(f"🔍 명사 추출 후 리뷰 수: {len(df_reviews_nouns)}")
    logger.debug(f"🔍 Noun extraction 예시: {df_reviews_nouns['content_nouns'].iloc[0]}")
except Exception as e:
    logger.error(f"❌ 명사 추출 실패: {e}")
    return

try:
    logger.info("✅ Step 6: 유효 리뷰만 필터링...")
    logger.debug(f"🔍 유효성 검증 전 리뷰 수: {len(df_reviews_nouns)}")
    df_reviews_valid = validate_reviews(df_reviews_nouns)
    logger.debug(f"🔍 유효성 검증 후 리뷰 수: {len(df_reviews_valid)}")
    logger.debug(f"🔍 유효 리뷰 수: {len(df_reviews_valid)}")
except Exception as e:
    logger.error(f"❌ 유효 리뷰 필터링 실패: {e}")
    return
```

리뷰 텍스트 클렌징



명사 추출 및 불용어 제거



유효 리뷰만 필터링

• 데이터 전처리 파이프라인

```

try:
    logger.info("📦 Step 7: JSON 변환 중...")
    place_info_json, reviews_json = convert_to_json(df_place_filtered, df_reviews_valid)
    logger.debug(f"📄 JSON 샘플 place_info: {place_info_json[0]}")
    logger.debug(f"📄 JSON 샘플 review: {reviews_json[0]}")
except Exception as e:
    logger.error(f"❌ JSON 변환 실패: {e}")
    return

try:
    logger.info("🌐 Step 8: 지역구 컬렉션 초기화...")
    init_collections(region_csv_path)
    logger.debug(f"📄 CSV 경로: {region_csv_path}")
except Exception as e:
    logger.error(f"❌ 지역구 컬렉션 초기화 실패: {e}")
    return

try:
    logger.info("📁 Step 9: MongoDB에 데이터 적재 중...")
    insert_data(place_info_json, reviews_json)
    logger.debug("✅ 데이터 적재 성공")
except Exception as e:
    logger.error(f"❌ MongoDB 데이터 적재 실패: {e}")
    return

logger.info("🎉 모든 데이터 파이프라인 완료 및 MongoDB 저장 완료!")

# 실행 (python pipeline.py)
if __name__ == "__main__":
    run_pipeline("data/adm_dong_list.csv")

```

DF를 JSON으로 변환



MongoDB 지역구 컬렉션 만들기



MongoDB에 place, review 적재

완료 !

• 데이터 전처리 코드

1. 장소 중복 제거 및 리뷰 정리

deduplicate_places.py

```
# preprocess/deduplicate_places.py
import pandas as pd

def deduplicate_places(df_place_info: pd.DataFrame, df_reviews: pd.DataFrame): 4 usages  Kaeun Han
    # 1. 중복 place_id 찾기
    duplicated_place_ids = df_place_info[df_place_info.duplicated(subset="place_id", keep=False)]["place_id"].unique().tolist()

    # 2. adm_dong_code 기준으로 정렬 후, 중복 제거 (adm_dong_code가 가장 작은 것만 남김)
    df_place_info_sorted = df_place_info.sort_values("adm_dong_code")
    df_place_info_dedup = df_place_info_sorted.drop_duplicates(subset="place_id", keep="first")

    # 3. reviews: dedup된 place_id 중 중복이었던 place_id에 대해 각각 100개씩만 남기기
    df_reviews_filtered = df_reviews[df_reviews["place_id"].isin(duplicated_place_ids)].copy()
    df_reviews_limited = (
        df_reviews_filtered
        .groupby("place_id")
        .head(100)
        .reset_index(drop=True)
    )

    # 4. 중복이 아니었던 place_id 리뷰 전부 유지
    non_duplicated_place_ids = set(df_place_info_dedup["place_id"]) - set(duplicated_place_ids)
    df_reviews_remaining = df_reviews[df_reviews["place_id"].isin(non_duplicated_place_ids)]

    # 5. 최종 리뷰 데이터 합치기
    df_reviews_final = pd.concat(objs=[df_reviews_limited, df_reviews_remaining], ignore_index=True)

    return df_place_info_dedup.reset_index(drop=True), df_reviews_final.reset_index(drop=True), duplicated_place_ids
```

중복 place_id 찾기



adm_dong_code가
가장 작은 곳에 남김



reviews에서도 해당 중복 찾아 제거

• 데이터 전처리 코드

2. 필요한 컬럼만 필터링

filter_columns.py

```
import pandas as pd

def filter_columns(df_place_info: pd.DataFrame, df_reviews: pd.DataFrame): 4 usages  👤 RyeongahLim
    df_place_filtered = df_place_info[[
        "place_id", "adm_dong_code", "name", "category", "address", "opening_hours", "naver_rating"
    ]].copy()

    df_reviews_filtered = df_reviews[["place_id", "visit_count", "content"]].copy()

    return df_place_filtered, df_reviews_filtered
```

분석에 활용될 컬럼 -
place_info: 컬럼 7개,
reviews : 컬럼 3개만 추출

• 데이터 전처리 코드

3. 리뷰 텍스트 클렌징

clean_text.py

```
import pandas as pd
import re

def clean_text(df_reviews: pd.DataFrame): 4 usages  Kaeun Han +1
    # 1. content 컬럼의 결측치 먼저 제거
    df_reviews = df_reviews.dropna(subset=["content"])

    # 2. 특수문자, 이모지 제거
    df_reviews.loc[:, "content"] = df_reviews["content"].apply(
        lambda x: re.sub(pattern=r"[^가-힣a-zA-Z0-9\s]", repl: "", str(x))
    )

    # 3. 공백만 남은 리뷰 제거 → 이건 "내용이 없다"는 거니까 제거
    df_reviews = df_reviews[df_reviews["content"].str.strip() != ""]

    # 4. 혹시 모르니 다시 한 번 결측치 제거 (위 처리로 인해 생겼을 수도 있는 NaN 방지)
    df_reviews = df_reviews.dropna(subset=["content"])

    return df_reviews.reset_index(drop=True)
```

리뷰가 결측치인 데이터 제거



특수문자, 이모지 제거



공백만 남은 리뷰 제거 (내용 없음)



위 과정들 이후에 생긴 결측치 최종 제거

• 데이터 전처리 코드

4. 명사 추출 및 불용어 제거

extract_nouns.py

```
import os
from konlpy.tag import Okt
from utils.logger import get_logger

logger = get_logger("extract_nouns")

okt = Okt() #형태소 분석기 for 명사추출

# korean_stopwords.txt 파일 경로 설정
STOPWORDS_PATH = os.path.join(os.path.dirname(__file__), "korean_stopwords.txt")

def load_stopwords(): 1 usage  ⤴ RyeongahLim
    try:
        with open(STOPWORDS_PATH, "r", encoding="utf-8") as f:
            stopwords = set(line.strip() for line in f if line.strip())
            return stopwords
    except FileNotFoundError:
        logger.error(f"Stopwords file not found at {STOPWORDS_PATH}")
        return set()

def extract_nouns_from_text(text, stopwords): 1 usage  ⤴ RyeongahLim +1
    if not isinstance(text, str):
        return []

    nouns = okt.nouns(text)
    filtered = [word for word in nouns if word not in stopwords and len(word) > 1]
    return filtered
```

Okt: 한국어 형태소 분석기

okt.nouns(text)로 명사 추출

korean_stopwords.txt : 한국어 불용어 처리 파일

```
def extract_nouns_from_reviews(df_reviews): 4 usages  ⤴ Kaeun Han +1
    logger.info("명사 추출 및 불용어 제거 시작")
    stopwords = load_stopwords()
    total = len(df_reviews)

    content_nouns = []

    for idx, row in df_reviews.iterrows():
        text = row["content"]
        nouns = extract_nouns_from_text(text, stopwords)
        content_nouns.append(nouns)

        # 💡 중간 진행상황 로그 (1000개 단위로)
        if (idx + 1) % 1000 == 0 or (idx + 1) == total:
            logger.debug(f"🔄 명사 추출 진행 중: {idx + 1}/{total}개 완료")

    df_reviews["content_nouns"] = content_nouns
    logger.info("명사 추출 및 불용어 제거 완료")
    return df_reviews
```

• 데이터 전처리 코드

5. 유효한 리뷰 필터링

valid_reviews.py

```
import pandas as pd

def validate_reviews(df_reviews: pd.DataFrame): 4 usages  👤 RyeongahLim
    df_valid = df_reviews[df_reviews["content_nouns"].apply(lambda x: len(x) > 4)]
    return df_valid.reset_index(drop=True)
```

전처리를 모두 수행했을 때,
명사형 단어의 수가 5개 이상 있는 (= 리뷰로 상황을 정의가능한)
리뷰 데이터만 남김.

• 데이터 전처리 결과

```

- 📦 Place info shape: (17157, 12), Reviews shape: (1726634, 8)
- 🔍 Step 2: 중복 place_id 처리 및 리뷰 필터링 중...
- ✏️ Deduplicated places: (8758, 12), Deduplicated reviews: (881678, 8)
- 📁 Step 3: 컬럼 필터링...
- 📄 Filtered place columns: ['place_id', 'adm_dong_code', 'name', 'category', 'address', 'opening_hours', 'naver_rating']
- 📄 Filtered review columns: ['place_id', 'visit_count', 'content']

```

Json File Data > (17157, 12)(place_info), (1726634, 8)(reviews)

장소 중복 제거 및 리뷰 정리 >> (8739, 12)(place_info), (881678,8)(reviews)

컬럼 필터링 >> (8739, 7)(place_info), (881678, 3)(reviews)

```

- 🌿 Step 4: 리뷰 텍스트 클렌징...
- 📄 클렌징 전 리뷰 수: 881678
- 📄 클렌징 후 리뷰 수: 826803
- ✂️ Cleaned review 예시: 오랜만에 파스타 맛집을 찾은거 같아 너무 좋았어요 일단 시금치 파스타는 바질과 시금치 향이 진해 너무 잘먹었습니다
  습니다문어 샐러드도 스타터로 입맛을 돋군게 딱이였습니다너무 오랜만에 잘 먹었습니다
- 🧠 Step 5: 명사 추출 및 불용어 제거...
- 📄 명사 추출 전 리뷰 수: 826803
  명사 추출 및 불용어 제거 시작

```

리뷰 텍스트 클렌징 >> 826730(reviews)

명사 추출 및 불용어 제거

• 데이터 전처리 결과

```

- ✅ Step 6: 유효 리뷰만 필터링...
- 📄 유효성 검증 전 리뷰 수: 826803
- 📄 유효성 검증 후 리뷰 수: 342873
- ✍ 유효 리뷰 수: 342873

```

유효한 리뷰 필터링 (≥ 5 nouns) >> 342873

```

- 📦 Step 7: JSON 변환 중...
- 📄 JSON 샘플 place_info: {'place_id': 36819674, 'adm_dong_code': 1111051500.0, 'name': '준수방키친', 'category': '이탈리아음식', 'ad
친', 'opening_hours': '월 11:30 - 21:30, 15:00 - 17:00 브레이크타임, 14:00, 20:30 라스트오더 화 11:30 - 21:30, 15:00 - 17:00 브레이크
이크타임, 14:00, 20:30 라스트오더 목 11:30 - 21:30, 15:00 - 17:00 브레이크타임, 14:00, 20:30 라스트오더 금 11:30 - 21:30, 15:00 - 17:0
20:30 라스트오더 일 11:30 - 20:30, 19:30 라스트오더', 'naver_rating': 4.43}
- 📄 JSON 샘플 review: {'place_id': 1534186654, 'visit_count': 1.0, 'content_nouns': ['파스타', '맛집', '시금치', '파스타', '바질', '사
리', '블루', '치즈', '소스', '기분', '문어', '샐러드', '스타', '입맛']}
- 🌐 Step 8: 지역구 컬렉션 초기화...
- 📁 CSV 경로: data/adm_dong_list.csv
- 📧 Step 9: MongoDB에 데이터 적재 중...
- ✅ 데이터 적재 성공
- 🎉 모든 데이터 파이프라인 완료 및 MongoDB 저장 완료!

```

MongoDB 적재 완료

- DB 정보 및 구성



de25proejct1DB +		
de25proejct1db.twmfiwj.mongodb.net > de25proejct1DB		
Sort by Collection Name		
place_info		
Storage size: 1.16 MB	Documents: 8.8 K	Avg. document size: 446.00 B
region_map		
Storage size: 28.67 kB	Documents: 425	Avg. document size: 95.00 B
reviews		
Storage size: 72.68 MB	Documents: 343 K	Avg. document size: 505.00 B

Database :
de25project1DB

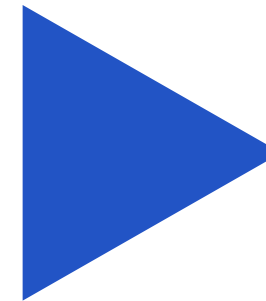
Collection 1 : place_info
Collection 2 : region_map
Collection 3 : reviews

• DB 정보 및 구성

region_map의 경우,
서울특별시 행정동 정보에 대한 csv를
하나의 collection으로 추가

	adm_dong_code	city	district	neighborhood
1				
2	1111051500	서울특별시	종로구	청운효자동
3	1111053000	서울특별시	종로구	사직동
4	1111054000	서울특별시	종로구	삼청동
5	1111055000	서울특별시	종로구	부암동
6	1111056000	서울특별시	종로구	평창동
7	1111057000	서울특별시	종로구	무악동
8	1111058000	서울특별시	종로구	교남동
9	1111060000	서울특별시	종로구	가회동
10	1111061500	서울특별시	종로구	종로1.2.3.4가동
11	1111063000	서울특별시	종로구	종로5.6가동
12	1111064000	서울특별시	종로구	이화동
13	1111065000	서울특별시	종로구	혜화동
14	1111067000	서울특별시	종로구	창신제1동
15	1111068000	서울특별시	종로구	창신제2동

adm_dong_list.csv



de25proejct1db.twmfiwj.mongodb.net > de25proejct1DB > region_map

Documents 425 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA EXPORT DATA UPDATE DELETE

```

_id: ObjectId('67fbc1e7c3fae86e7548e50f')
adm_dong_code : 1111051500
district : "종로구"
neighborhood : "청운효자동"

_id: ObjectId('67fbc1e7c3fae86e7548e512')
adm_dong_code : 1111055000
district : "종로구"
neighborhood : "부암동"

_id: ObjectId('67fbc1e7c3fae86e7548e518')
adm_dong_code : 1111063000
district : "종로구"
neighborhood : "종로5.6가동"

_id: ObjectId('67fbc1e7c3fae86e7548e51e')
adm_dong_code : 1111070000
district : "종로구"
neighborhood : "송인제1동"

```

collection : region_map

• 최종 DB 정보 및 구성

place_info

<필드 구성>

_id(ObjectId)
place_id(INT32)
adm_dong_code(Double)
name(String)
category(String)
address(String)
opening_hours(String)
naver_rating(Double)

음식점 정보를 담은 Collection

region_map

<필드 구성>

_id(ObjectId)
adm_dong_code(INT32)
district(String)
neighborhood(String)

지역구 / 행정동을 담은 Collection

reviews

<필드 구성>

_id(ObjectId)
place_id(INT32)
visit_count(Double)
content_nouns(Array)
situation_definition(String)
situation_cluster(INT32)
cluster_name(String)

음식점별 리뷰를 담은 Collection

- **MongoDB 인덱스 최적화**

검색 성능 향상을 위한 indexes 추가

컬렉션	인덱스 필드	목적
place_info	adm_dong_code	행정동 기반 빠른 검색 가속화
place_info	place_id	place_id로의 빠른 검색 가속화
reviews	place_id, situation_cluster	클러스터와 음식점에 따른 리뷰 집계 가속화

05

데이터 분석

• LLM 기반 상황정의 생성 파이프라인



• LLM 결정 과정 (Claude)

고려사항 : 대규모 리뷰 데이터를 처리하면서도
문맥에 자연스럽게, 개연성 있는 상황 정의를 생성하는 것이 중요

OpenAI GPT 4o-mini

Google Gemini pro

Hugging Face Models

PracticeLLM/
Custom-KoLLM-13B-v7

Claude 3.5 Haiku

✓ 비용 + 속도 측면 선택

• KoSBERT 모델

KoSBERT 정보: <https://huggingface.co/jhgan/ko-sbert-nli>

문장형 임베딩 계산 (Sentence-BERT 구조)를 한국어에 fine-tuning한 모델

각 상황정의 문장들을 클러스터링하기 위해 유사도 계산에 활용

• Claude 3.5 Haiku from Antrophic

Credit balance

Your credit balance will be consumed with API, Claude Code and Workbench usage. You can buy credits directly or set up auto-reload thresholds.

AI

US\$19.42

Remaining Balance

Charged to

link

Link by Stripe↻

Buy credits

⊗

Auto reload is disabled. Enable auto reload to avoid API interruptions when credits are fully spent.

Edit

돈을 충전해놓고 금액 안에서
API 요청을 보낼 수 있는 시스템

API keys 1

API keys are owned by workspaces and remain active even after the creator is removed

KEY	WORKSPACE	CREATED BY	CREATED AT ^
deproject1-onbo... sk-ant-api03-min...SAAA	Default ⓘ	DEproject1 de25project1@gmail.com	Apr 20, 2025

Apr 21, 2025	Credit grant	Expiring 4/22/2026	US\$44.00
Apr 20, 2025	Credit grant	Expiring 4/21/2026	US\$38.50
Apr 20, 2025	Credit grant	Expiring 4/21/2026	US\$5.50

- 프롬프트 설계 시도 과정

여러 시도를 거치면서 입력 토큰수를 줄이면서도,
결과가 원하는 방식으로 나오도록 지속적으로 도전

각 리뷰에 대한 상황정의 생성은 **6차 시도** 끝에 최종 프롬프트 완성

각 클러스터에 대표 상황 이름 생성은 **2n차 시도** 끝에 최종 프롬프트 완성

각 시도에 대한 프롬프트와 결과는 노선에 수시로 기록

프롬프트 설계 시도 과정

1차 시도

```
# prompt
prompt = (
    "아래 키워드를 참고해서, 어떤 상황에 이런 장소를 추천할 수 있을지 한 문장:
    "문장은 '~할 때'로 끝나는 형식으로 작성해줘.\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "상황정의:"
)
return prompt
```

- 프롬프트에 맞지 않음. ~때로 끝나지 않음.
- 응답시간, 처리시간이 중복으로 뜸.
- 응답과 상황정의가 중복으로 뜸.

2차 시도

```
# prompt
prompt = (
    "다음 키워드를 참고해서, 어떤 상황에 이런 장소를 추천할 수 있을지 한 문장으로 정의해
    "문장은 반드시 '~할 때'로 끝나야 해. 예: '가족과 함께 편안한 식사를 즐기고 싶을 때'
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

- 처리 시간이 많이 늘어남.

3차 시도

- 처리 시간은 줄었으나, ~때, ~때가 두번씩 나오는 경우가 있음. 한 문장이라고 했음에도 불구하고.

Python ▾

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 말해줘.\n"
    "문장은 '~할 때'로 끝내.\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

4차 시도 : 더 명확하게 '하나만', '한 문장'을 강조

- 나름 괜찮음. 처리시간도 그렇고. 그렇지만, 문장이 조금 긴 게 문제.

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 말해줘.\n"
    "문장은 '~할 때'로 끝내.**문장은 하나만 출력해.**\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

5차 시도

- "~레스토랑입니다."라고 나오는 경우가 생김. 도대체 왜 내 프롬프트를 무시하는가.

```
# prompt
prompt = (
    "다음 키워드를 참고해, 이 장소를 어떤 상황에서 추천할 수 있을지 **간결한
    "반드시 '~할 때'로 끝나야 해. **하나의 상황만 말해줘.**\n\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```

★ 6차 시도 : 4차에서 "간결하게"라는 표현만 추가. (완성 !!) ★

- 문장도 짧게 나오고, '~할 때'로 나옴.

```
# prompt
prompt = (
    "다음 키워드로 유추되는 장소 추천 상황을 한 문장으로 간결하게 말해줘.\n"
    "문장은 '~할 때'로 끝내.**문장은 하나만 출력해.**\n"
    f"키워드: {' ', ' '.join(content_nouns)}\n"
    "정의 문장:"
)
return prompt
```


- 프롬프트 결정

```
@staticmethod 1 usage  👤 Kaeun Han
def build_prompt(content_nouns: List[str]) -> str:
    prompt = (
        "다음 키워드로 유추되는 음식점 추천 상황을 한 문장으로 간결하게 말해줘.\n"
        "문장은 '~할 때'로 끝내. 문장은 하나만 출력해.\n"
        f"키워드: {' '.join(content_nouns)}\n"
        "상황 정의:"
    )
    return prompt
```

Step1.

각 리뷰에 대한 상황정의를
생성하는 프롬프트

• 프롬프트 결정

```
class ClaudeAPIWrapper: 2 usages  Kaeun Han

def generate_label(self, situation_definitions_batch): 1 usage  Kaeun Han
    joined_text = "\n".join(f"- {s}" for s in situation_definitions_batch)

    prompt = (
        "다음 문장들을 보고 사용자가 어떤 상황일 때 해당 음식점을 방문하는 것이 좋을지, 하나의 주제 문장을 만들어줘.\n"
        "\n"
        "조건:\n"
        "- 문장은 반드시 '~할 때'로 끝내.\n"
        "- 음식 종류나 메뉴 위주로 작성하지마. 대신 기분, 추천 상황, 목적 같은 특징은 살려서 표현해줘.\n"
        "- 구체적인 욕구나 활동(예: 스트레스 풀고 싶을 때, 기름진 음식을 즐기고 싶을 때)로 표현해줘.\n"
        "- 문장 속 단어를 그대로 나열하는 식이 아니었으면 좋겠어.\n"
        "- 하나의 공통된 패턴과 의미를 반영해.\n"
        "\n"
        "예시:\n"
        "- 친구랑 수다 떨며 편하게 모이고 싶을 때\n"
        "- 비 오는 날 따뜻한 국물 요리를 즐기고 싶을 때\n"
        "\n"
        f"{joined_text}\n\n대표 문장:"
    )
```

Step3.

각 클러스터에 대한
대표 상황 이름을
생성하는 프롬프트

• Challenges : API 토큰 제한 해결

Step1.

각 리뷰의 상황정의 생성

리뷰의 수가 많음.
각 리뷰의 토큰수는 적음.



asyncio 비동기 처리
batch size,
concurrency_limit,
max_rps 설정

Step3.

각 클러스터의 대표 상황명 생성

클러스터 개수는 적음.
각 클러스터에 해당된
토큰수 매우 많음.



동기 처리
각 클러스터에 속한
상황정의 batch size

• KoSBERT + 클러스터링 파이프라인

```
# 3. 전체 실행 흐름
if __name__ == "__main__":
    # (1) 로딩
    print("🚀 merged_vectors.json 로딩 중...")
    ids, embeddings = load_vectors_from_merged_json("./step2_clustering/merged_vectors.json")
    print(f"✅ 벡터 수: {len(ids)}")

    # (2) KMeans 클러스터링
    print("🔍 KMeans 클러스터링 중...")
    kmeans = KMeans(n_clusters=20, random_state=42, n_init="auto")
    labels = kmeans.fit_predict(embeddings)

    id_to_cluster = {id_: int(cluster) for id_, cluster in zip(ids, labels)}

    # (3) 품질 평가
    print(f"📊 관성(Inertia): {kmeans.inertia_:.2f}")
    sil_score = silhouette_score(embeddings, labels)
    print(f"🇮🇹 실루엣 점수: {sil_score:.4f}")

    # (4) MongoDB 업로드
    print("📡 MongoDB에 클러스터 결과 업로드 중...")
    upload_cluster_results_to_mongo(id_to_cluster)
```

KoSBERT 모델을 돌린 후,
merged_vectors.json 파일을 저장

merged_vectors.json 파일 로드

KMeans Clustering(k=20)

MongoDB에 클러스터링 결과 업로드
(situation_cluster 필드)

• 대표 상황 20개

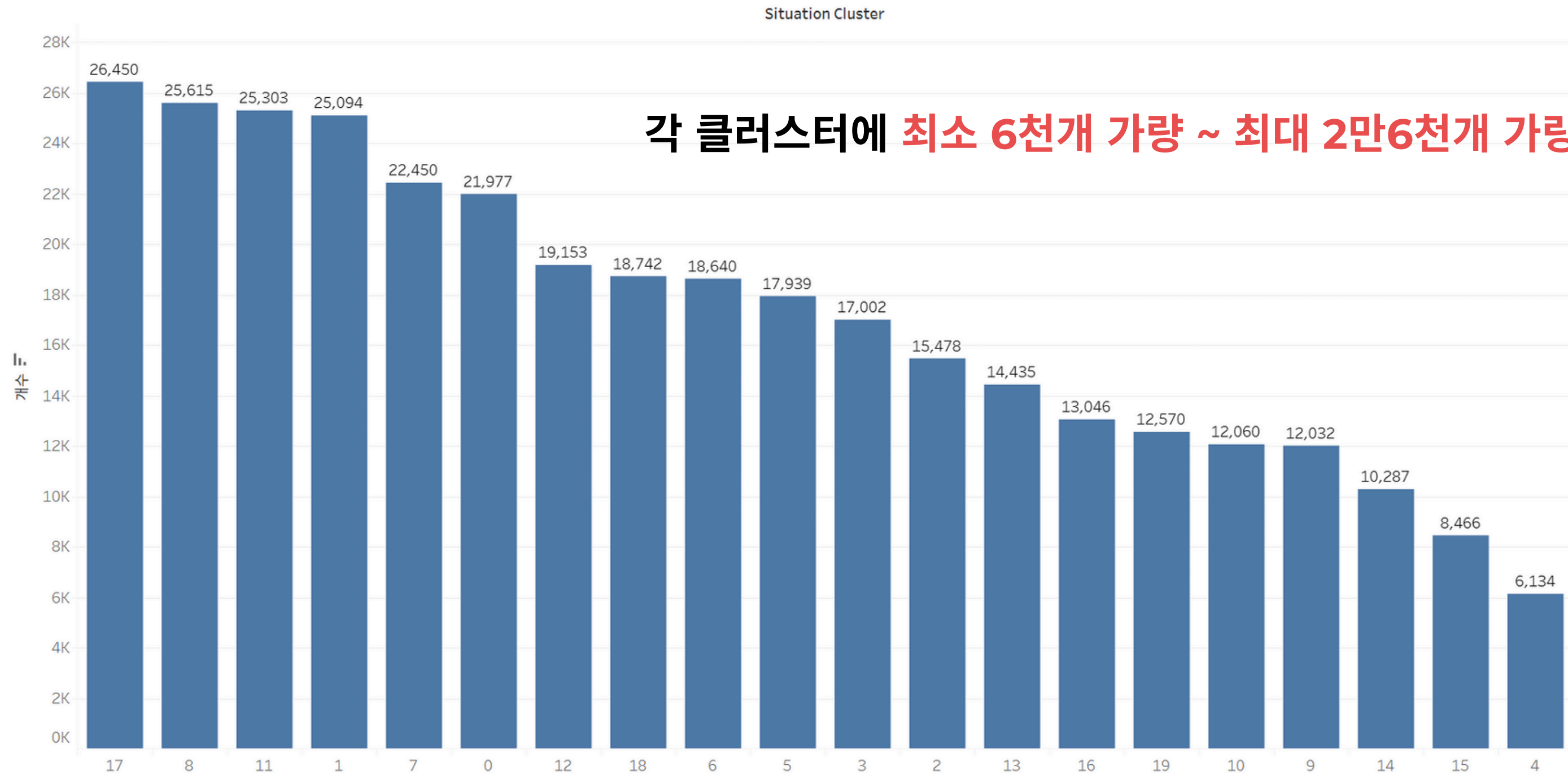
- 0 스트레스 해소를 위해 매운 음식이 땡길 때
- 1 혼밥에 부담없는 맛집을 찾을 때
- 2 식사 메뉴에 고기가 빠지면 아쉬운 사람일 때
- 3 배달이나 포장에 편리한 식당이 필요할 때
- 4 곱창과 다양한 내장요리를 맛있게 즐기고 싶을 때
- 5 **뜨끈한 국물로 몸을 녹이고 싶을 때**
- 6 친구나 가족과 함께 소중한 시간을 보내고 싶을 때
- 7 친절함 서비스로 기분 좋은 식사시간을 원할 때
- 8 **집밥처럼 정성가득한 반찬을 맛보고 싶을 때**
- 9 정통 일본 요리를 음미하고 싶을 때

- 10 **맛있는 안주가 있는 술집을 찾을 때**
- 11 고기의 맛과 질에 만족하는 식사를 하고 싶을 때
- 12 가족들과 주말에 방문할 맛집을 찾을 때
- 13 데이트 코스로 양식을 대접하고 싶을 때
- 14 특별한 날에 만족할 만한 서비스를 원할 때
- 15 다양한 중식 메뉴가 있는 분위기 좋은 중식당을 찾을 때
- 16 친구들과 편하게 치맥을 즐기고 싶을 때
- 17 가성비 좋은 식당에서 지인들과 모임을 가지고 싶을 때
- 18 편안한 분위기에서 여유롭게 식사하며 기분 전환하고 싶을 때
- 19 **신선한 수산물 요리를 괜찮은 가격에 즐기고 싶을 때**

• 대표 상황 20개

클러스터별 문서 개수 확인 차트

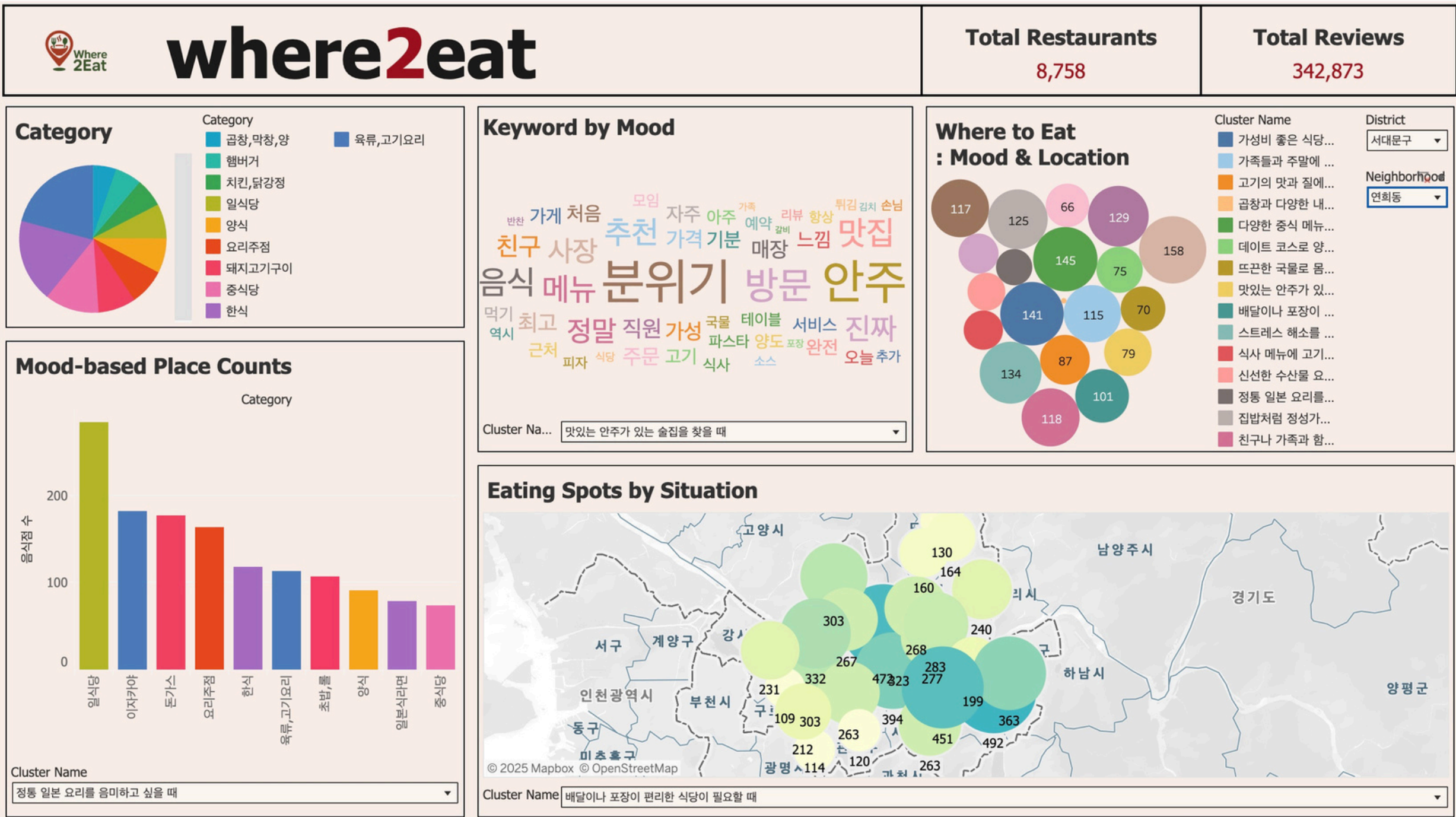
클러스터별 document 개수



06

데이터 시각화

• 데이터 시각화 (with Tableau)



- Total Restaurants
 - 총 음식점 수
- Total Reviews
 - 총 리뷰 수
- Category
 - 각 카테고리별 음식점 수
- Keyword by Mood
 - 상황별 키워드 워드클라우드
- Where to Eat: Mood & Location
 - 지역별 상황 수 버블차트
- Mood-based Place Counts
 - 상황별 카테고리 수 막대그래프
- Eating Spots by Situation
 - 상황별 빈도수 지도

07

서비스 구현

• 사이트 기획 (with Figma)



메인페이지
원하는 지역구를 선택하는 화면

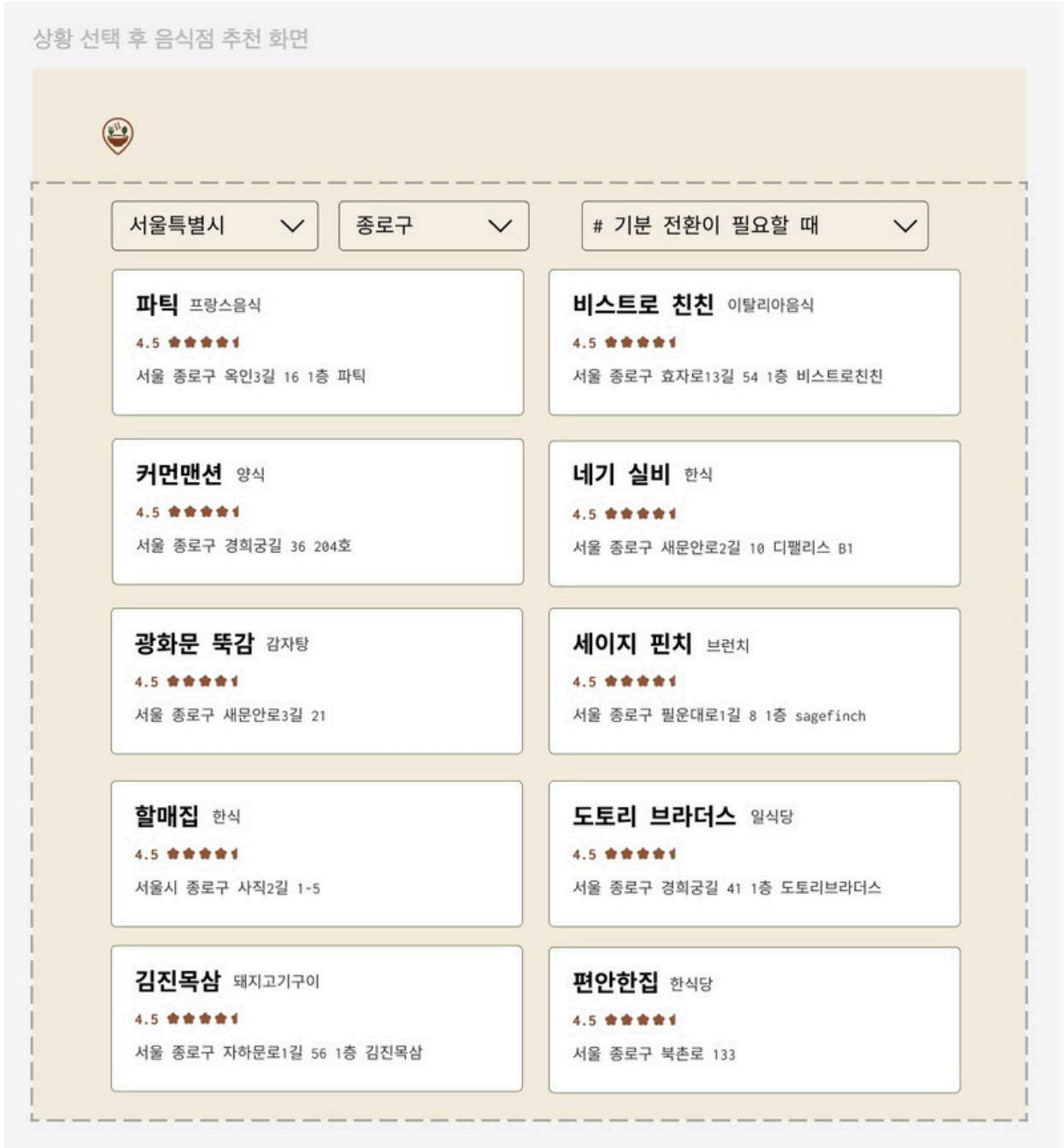


로딩페이지
해당 지역구의 버블차트로 가는 로딩 화면

• 사이트 기획 (with Figma)



선택페이지
상황에 대한 버블차트에서 선택하는 화면



추천페이지
음식점에 대한 추천이 뜨는 화면

• 사이트 개발

프론트엔드: **React(JS)**, **Vite**, **D3.js**, React Router DOM

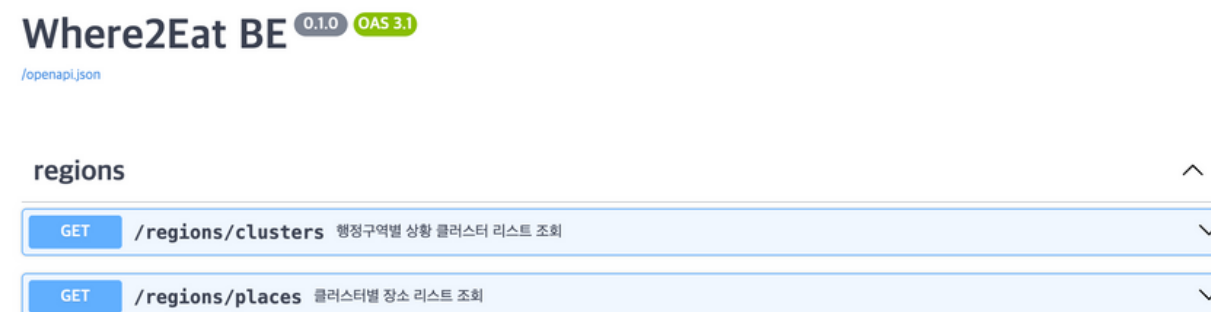
- GitHub: <https://github.com/DE-Project1/situation-restaurant-ui-react>
- 배포 웹사이트 링크: <https://scintillating-sable-e44ff7.netlify.app/>



D3.js : JS기반 데이터 시각화 라이브러리 → **버블차트** 개발

백엔드: **FastAPI(Python)**, **Uvicorn**, **Pymongo**, AWS EC2, Docker, GitHub Actions

- GitHub: <https://github.com/DE-Project1/Where2Eat-BE>
- 배포 서버 API 문서: <https://api.where2eat.r-e.kr/docs>

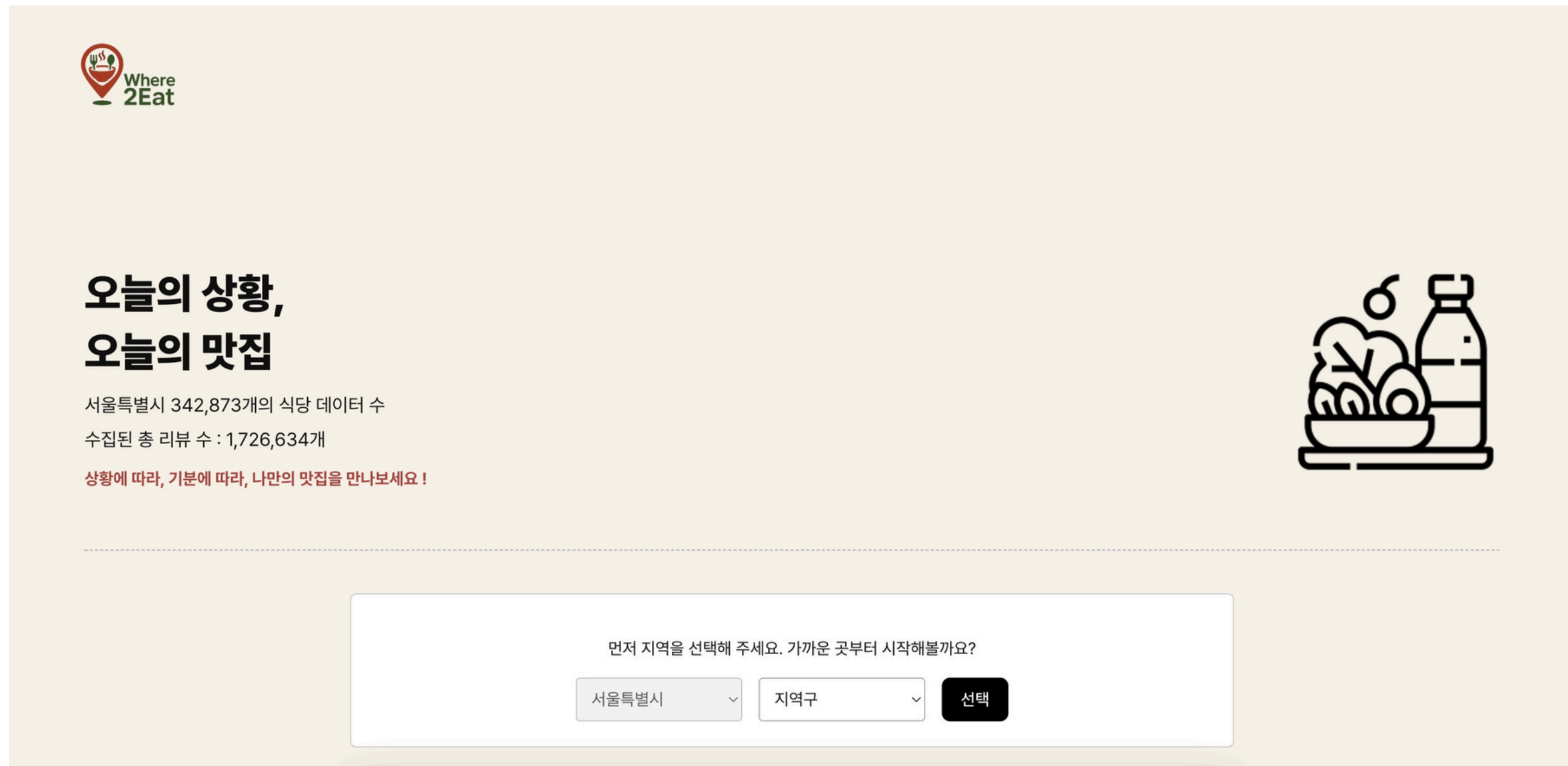


MongoDB 연결, 데이터 꺼내 FE에 전달하는 API 2개 개발

• 사이트 제작 결과물



웹사이트 로고 제작 with GPT



[Where2Eat 웹사이트 연결 링크](#)

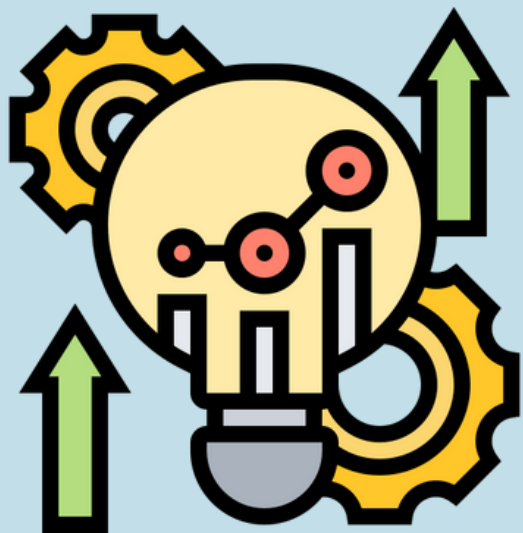
- 사이트 demo



웹사이트 데모 영상

현재 서버를 중단한 상태로,
데모 영상으로 대체합니다.

• Further Developments



1. 사용자 맞춤형 추천 가능성 확대

- 클러스터링 없이 각 리뷰의 상황 정의를 그대로 활용
- Elastic Search 혹은 Pinecone과 같은 기술

2. 서비스 범위 확장

- 서울 → 전지역으로 서비스 확대
- 실시간 정보 업데이트로 시의성 있는 추천 제공
- 데이터 처리 용량 대응 및 API 연동 고도화 필요

Thank You